

SSCHA School Tutorials

Hands-on sessions on the Stochastic Self-Consistent Harmonic Approximation

Contents

1	Hands-on Session 1 - First SSCHA Simulations: Free Energy and Structural Relaxations	3
1.1	The Free Energy of Gold: A Simulation in the NVT Ensemble	3
1.2	Plot the Phonon Dispersion	8
1.3	Running in the NPT Ensemble: Simulating Thermal Expansion	10
1.4	Ab Initio Calculation with the SSCHA Code	13
1.5	Parallelization	16
2	Hands-on Session 3 - Calculations of Second-Order Phase Transitions with the SSCHA	18
2.1	Structural Instability: Calculation of the Hessian	18
2.2	Temperature Dependence of the Free Energy Hessian	20
2.3	Analysis	21
2.4	Interpretation	21
2.5	Exercise	22
2.6	Second Order Phase Transition	26
3	Hands-on Session 3 - Spectral functions, Raman and infrared spectra with the Time-Dependent SSCHA	31
3.1	Theoretical background	31
3.1.1	The phonon Green's function	33
3.2	Tutorial	33
3.2.1	Deep analysis of the script	35
3.2.2	Extract the green's function and plot	38
3.3	Raman Response	41
3.3.1	Unpolarized Raman	45
4	Hands-on Session 9: Thermal Conductivity Calculations with the SSCHA	47
4.1	Lattice Thermal Conductivity of CsPbI ₃	47
4.2	Exercise	57
5	Hands-on-session 5: EPIq - Anharmonicity in electron-phonon coupling related properties	58
5.1	Introduction	58
5.2	Requirements	58
5.3	About EPIq	59
5.3.1	Calculations available in EPIq	59
5.3.2	EPIq workflow	61
5.3.3	EPIq input file	61
5.4	Setup phase: how to setup a calculation with EPIq	61
5.4.1	Wannier interpolation	61
5.4.2	Check real space localization	62
5.5	Exercise phase: calculation of electron-phonon coupling related properties for doped monolayer MoS ₂	62

5.6	Solution: Phonon linewidth calculation	64
5.7	Advanced tutorial: Migdal-Eliashberg calculation using SSCHA Hessian matrices	68

Chapter 1

Hands-on Session 1 - First SSCHA Simulations: Free Energy and Structural Relaxations

In this hands-on session, we provide ready-to-use examples to set up your first SSCHA calculation.

1.1 The Free Energy of Gold: A Simulation in the NVT Ensemble

This simple tutorial explains how to set up an SSCHA calculation starting from just the structure, in this case a CIF file downloaded from the [Materials Project](#) database, which we can find in the `01_First_SSCHA_simulations` directory.

Starting from the Gold structure in the primitive cell, to run the SSCHA we need:

1. Compute the harmonic phonons (dynamical matrix)
2. Remove imaginary frequencies (if any)
3. Run the SSCHA

At the very beginning, we simply import the SSCHA libraries, CellConstructor, the math libraries, and the force field. This is done in Python with the `import` statements.

```
1 # Import the sscha code
2 import sscha, sscha.Ensemble, sscha.SchaMinimizer
3 import sscha.Relax, sscha.Utilities
4
5 # Import the cellconstructor library to manage phonons
6 import cellconstructor as CC, cellconstructor.Phonons
7 import cellconstructor.Structure, cellconstructor.calculators
8
9 # Import the force field of Gold
10 import ase, ase.calculators
11 from ase.calculators.emt import EMT
12
13 # Import numerical and general purpose libraries
14 import numpy as np, matplotlib.pyplot as plt
15 import sys, os
```

The first thing we do is initialize a CellConstructor structure from the CIF file downloaded from the material database (`Au.cif`). We initialize the EMT calculator from ASE and relax the structure:

```

1 gold_structure = CC.Structure.Structure()
2 gold_structure.read_generic_file("Au.cif")
3
4 # Get the force field for gold
5 calculator = EMT()
6
7 # Relax the gold structure
8 relax = CC.calculators.Relax(gold_structure, calculator)
9 gold_structure_relaxed = relax.static_relax()

```

In the case of Gold, the relaxation is unnecessary, as it is an FCC structure with the Fm-3m symmetry group and 1 atom per primitive cell. This means the atomic positions have no degrees of freedom, so the relaxation will end before it even starts.

Next, we perform the harmonic phonon calculation using CellConstructor and a finite displacement approach:

```

1 gold_harmonic_dyn = CC.Phonons.compute_phonons_finite_displacements(
2     gold_structure_relaxed, calculator, supercell = (4,4,4))
3
4 # Impose the symmetries and
5 # save the dynamical matrix in the quantum espresso format
6 gold_harmonic_dyn.Symmetrize()
7 gold_harmonic_dyn.save_qe("gold_harmonic_dyn")

```

The method `compute_phonons_finite_displacements` is documented in the CellConstructor guide. It requires the structure (in this case `gold_structure_relaxed`), the force field (`calculator`), and the supercell for the calculation. Here we use a 4x4x4 supercell (equivalent to 64 atoms). This may not be sufficient to converge all properties, especially at very high temperature, but it is a good starting point.

Note that `compute_phonons_finite_displacements` works in parallel with MPI. Therefore, if the script is executed with `mpirun -np 16 python myscript.py`, it will split the calculations of the finite displacements across 16 processors. You need to have `mpi4py` installed. However, in this case, due to the high symmetry, only one calculation is required to obtain the harmonic dynamical matrix, so parallelization is unnecessary.

After computing the harmonic phonons in `gold_harmonic_dyn`, we impose the correct symmetrization and the acoustic sum rule with the `Symmetrize` method, and save the result in the Quantum ESPRESSO format with `save_qe`.

We are ready to submit the SSCHA calculation in the NVT ensemble.

The important parameters are:

- The temperature
- The number of random configurations in the ensemble
- The maximum number of iterations

These parameters are almost self-explanatory. In contrast with Molecular Dynamics (MD) or Metropolis-Monte Carlo (MC) calculations, where the equilibrium probability distribution is sampled through a dynamical evolution of the structure, the SSCHA encodes the whole probability distribution as an analytical function. Therefore, to compute properties, we can generate on the fly the configurations that sample the equilibrium distribution.

Workflow of the SSCHA objects for a free energy minimization.

Figure 1.1: Workflow of the SSCHA objects for a free energy minimization.

The code that sets up and performs the SSCHA is the following:

```

1 TEMPERATURE = 300

```

```

2 N_CONFIGS = 50
3 MAX_ITERATIONS = 20
4
5 # Initialize the random ionic ensemble
6 ensemble = sscha.Ensemble.Ensemble(gold_harmonic_dyn, TEMPERATURE)
7
8 # Initialize the free energy minimizer
9 minim = sscha.SchaMinimizer.SSCHA_Minimizer(ensemble)
10 minim.set_minimization_step(0.01)
11
12 # Initialize the NVT simulation
13 relax = sscha.Relax.SSCHA(minim, calculator, N_configs = N_CONFIGS,
14 max_pop = MAX_ITERATIONS)
15
16 # Define the I/O operations
17 # To save info about the free energy minimization after each step
18 ioinfo = sscha.Utilities.IOInfo()
19 ioinfo.SetupSaving("minim_info")
20 relax.setup_custom_functions(custom_function_post = ioinfo.CFP_SaveAll)
21
22 # Run the NVT simulation
23 relax.relax(get_stress = True)
24
25 # Save the final dynamical matrix
26 relax.minim.dyn.save_qe("sscha_T300_dyn")

```

In the previous code we defined the main objects needed to run the simulation:

- `ensemble` (`sscha.Ensemble.Ensemble`) represents the ensemble of ionic configurations. We initialize it with the dynamical matrix (which determines how much atoms fluctuate around the centroids) and the temperature.
- `minim` (`sscha.SchaMinimizer.SSCHA_Minimizer`) performs the free energy minimization. It contains all the information regarding the minimization algorithm, such as the initial timestep (set to 0.01 here). You can avoid setting the timestep, as the code will automatically guess the best value.
- `relax` (`sscha.Relax.SSCHA`) automates the generation of ensembles, calculation of energies and forces, and the free energy minimization to perform an NVT or NPT calculation. To initialize it, we pass the `minim` (which contains the ensemble with the temperature), the force field (`calculator`), the number of configurations `N_configs`, and the maximum number of iterations.

In this example, most of the time is spent in the minimization. However, if we replace the force field with ab initio DFT, the time to run the minimization is negligible compared to the time required to compute energies and forces on the ensemble configurations. The total (maximum) number of energy/forces calculations is equal to the number of configurations times the number of iterations (passed through the `max_pop` argument).

The calculation is submitted with `relax.relax()`. However, before running the calculation we introduce another object, the `IOInfo`. This tells `relax` to save information about the free energy, its gradient, and the anharmonic phonon frequencies during the minimization in the files `minim_info.dat` and `minim_info.freqs`. It is not mandatory to use them, but it is very useful as it allows visualization of the minimization while it is running.

The full input file is:

```

1 # Import the sscha code
2 import sscha, sscha.Ensemble, sscha.SchaMinimizer, sscha.Relax, sscha.Utilities
3
4 # Import the cellconstructor library to manage phonons
5 import cellconstructor as CC, cellconstructor.Phonons
6 import cellconstructor.Structure, cellconstructor.calculators
7

```

```

8 # Import the force field of Gold
9 import ase, ase.calculators
10 from ase.calculators.emt import EMT
11
12 # Import numerical and general purpose libraries
13 import numpy as np, matplotlib.pyplot as plt
14 import sys, os
15
16
17 """
18 Here we load the primitive cell of Gold from a cif file.
19 And we use CellConstructor to compute phonons from finite differences.
20 The phonons are computed on a q-mesh 4x4x4
21 """
22
23 gold_structure = CC.Structure.Structure()
24 gold_structure.read_generic_file("Au.cif")
25
26 # Get the force field for gold
27 calculator = EMT()
28
29 # Relax the gold structure (useless since for symmetries it is already relaxed)
30 relax = CC.calculators.Relax(gold_structure, calculator)
31 gold_structure_relaxed = relax.static_relax()
32
33 # Compute the harmonic phonons
34 # NOTE: if the code is run with mpirun, the calculation goes in parallel
35 gold_harmonic_dyn = CC.Phonons.compute_phonons_finite_displacements(
36     gold_structure_relaxed, calculator, supercell = (4,4,4))
37
38 # Impose the symmetries and
39 # save the dynamical matrix in the quantum espresso format
40 gold_harmonic_dyn.Symmetrize()
41 gold_harmonic_dyn.save_qe("harmonic_dyn")
42
43 # If the dynamical matrix has imaginary frequencies, remove them
44 gold_harmonic_dyn.ForcePositiveDefinite()
45
46 """
47 gold_harmonic_dyn is ready to start the SSCHA calculation.
48
49 Now let us initialize the ensemble, and the calculation at 300 K.
50 We will run a NVT calculation, using 100 configurations at each step
51 """
52
53 TEMPERATURE = 300
54 N_CONFIGS = 50
55 MAX_ITERATIONS = 20
56
57 # Initialize the random ionic ensemble
58 ensemble = sscha.Ensemble.Ensemble(gold_harmonic_dyn, TEMPERATURE)
59
60 # Initialize the free energy minimizer
61 minim = sscha.SchaMinimizer.SSCHA_Minimizer(ensemble)
62 minim.set_minimization_step(0.01)
63
64 # Initialize the NVT simulation
65 relax = sscha.Relax.SSCHA(minim, calculator, N_configs = N_CONFIGS,

```

```

66 max_pop = MAX_ITERATIONS)
67
68 # Define the I/O operations
69 # To save info about the free energy minimization after each step
70 ioinfo = sscha.Utilities.IOInfo()
71 ioinfo.SetupSaving("minim_info")
72 relax.setup_custom_functions(custom_function_post = ioinfo.CFP_SaveAll)
73
74
75 # Run the NVT simulation (save the stress to compute the pressure)
76 relax.relax(get_stress = True)
77
78 # Save the final dynamical matrix
79 # And print in stdout the info about the minimization
80 relax.minim.finalize()
81 relax.minim.dyn.save_qe("sscha_T{}_dyn".format(TEMPERATURE))

```

Now save the file as `sscha_gold.py` and execute it with:

```

1 $ python sscha_gold.py > output.log

```

And that is it. The code will probably take a few minutes on a standard laptop computer.

Congratulations! You ran your first SSCHA simulation!

If you open a new terminal in the same directory as the SSCHA submission, you can plot the information during the minimization. Starting from version 1.2, we provide visualization utilities installed together with the SSCHA. Simply type:

```

1 $ sscha-plot-data.py minim_info

```

You will see two windows.

Minimization data of Gold.

Figure 1.2: Minimization data of Gold.

The figure above shows all the minimization data. In the top-left panel, we see the free energy. As expected, it decreases (since the SSCHA is minimizing it). You can see that at certain values of the steps there are discontinuities. These occur when the code realizes that the ensemble on which it is computing is no longer good and a new one is generated. The quality of an ensemble is determined by the Kong-Liu effective sample size (bottom-left). When it reaches 0.5 of its initial value (equal to the number of configurations), the ensemble is extracted again and a new iteration starts. You can see that in the last iteration, the code stops before getting to 25 ($0.5 \cdot 50$). This means that the code converged properly: the gradient reached zero while the ensemble was still good.

On the right side you see the free energy gradients, which must go to zero to converge. The top-right is the gradient of the SSCHA dynamical matrix, while on bottom-right there is the gradient of the average atomic positions. Indeed, since the gold atomic positions are all fixed by symmetries, it is always zero (but it will be different from zero in more complex systems).

All the SSCHA phonon frequencies as a function of the step in the NVT simulation.

Figure 1.3: All the SSCHA phonon frequencies as a function of the step in the NVT simulation.

The figure above represents the evolution of the SSCHA phonon frequencies. Here, all the frequencies in the supercell (at each q-point commensurate with the calculation) are shown.

Note:

The SSCHA auxiliary frequencies in the figure above are not the real frequencies observed in experiments, but are rather linked to the average displacements of atoms along that mode.

By looking at how they change, you can get an idea of which phonon modes are more affected by anharmonicity. In this case, it is evident that Gold is strongly anharmonic and that the temperature makes all the phonon frequencies harder.

At the end of the simulation, the code writes the final dynamical matrix in the Quantum ESPRESSO file format: `sscha_T300_dynX` where X goes over the number of irreducible q-points.

In the next section, we analyze in detail each section of the script to provide a bit more insight into the simulation, and a guide to modify it to fit your needs and submit your own system.

1.2 Plot the Phonon Dispersion

Now that the SSCHA minimization has ended, we can compare the harmonic and anharmonic phonon dispersion of Gold.

For this purpose, we can simply run a script like the following. You can find a copy of this script already in `Examples/ThermodynamicsOfGold/plot_dispersion.py`. You can use it even in your own simulation; simply edit the value of the uppercase keyword at the beginning of the script to match your needs.

```
1  # Import the CellConstructor library to plot the dispersion
2  import cellconstructor as CC, cellconstructor.Phonons
3  import cellconstructor.ForceTensor
4
5  # Import the numerical libraries and those for plotting
6  import numpy as np
7  import matplotlib.pyplot as plt
8
9  import sys, os
10
11 # Let us define the PATH in the brilluin zone and the total number of points
12 PATH = "GXWXKGL"
13 N_POINTS = 1000
14
15 # Here we define the position of the special points
16 SPECIAL_POINTS = {"G": [0,0,0],
17                   "X": [0, .5, .5],
18                   "L": [.5, .5, .5],
19                   "W": [.25, .75, .5],
20                   "K": [3/8., 3/4., 3/8.]}
21
22 # The two dynamical matrix to be compared
23 HARM_DYN = 'harmonic_dyn'
24 SSCHA_DYN = 'sscha_T300_dyn'
25
26 # The number of irreducible q points
27 # i.e., the number of files in which the phonons are stored
28 NQIRR = 13
29
30
31 # ----- THE SCRIPT FOLLOWS -----
32
33 # Load the harmonic and sscha phonons
34 harmonic_dyn = CC.Phonons.Phonons(HARM_DYN, NQIRR)
```

```

35 sscha_dyn = CC.Phonons.Phonons(SSCHA_DYN, NQIRR)
36
37 # Get the band path
38 qpath, data = CC.Methods.get_bandpath(harmonic_dyn.structure.unit_cell,
39     PATH,
40     SPECIAL_POINTS,
41     N_POINTS)
42 xaxis, xticks, xlabel = data # Info to plot correctly the x axis
43
44 # Get the phonon dispersion along the path
45 harmonic_dispersion = CC.ForceTensor.get_phonons_in_qpath(harmonic_dyn, qpath)
46 sscha_dispersion = CC.ForceTensor.get_phonons_in_qpath(sscha_dyn, qpath)
47
48 nmodes = harmonic_dyn.structure.N_atoms * 3
49
50 # Plot the two dispersions
51 plt.figure(dpi = 150)
52 ax = plt.gca()
53
54 for i in range(nmodes):
55     lbl=None
56     lblsscha = None
57     if i == 0:
58         lbl = 'Harmonic'
59         lblsscha = 'SSCHA'
60
61     ax.plot(xaxis, harmonic_dispersion[:,i], color='k',
62         ls='dashed', label=lbl)
63     ax.plot(xaxis, sscha_dispersion[:,i],
64         color='r', label=lblsscha)
65
66 # Plot vertical lines for each high symmetry points
67 for x in xticks:
68     ax.axvline(x, 0, 1, color = "k", lw = 0.4)
69 ax.axhline(0, 0, 1, color = 'k', ls = ':', lw = 0.4)
70
71 # Set the x labels to the high symmetry points
72 ax.set_xticks(xticks)
73 ax.set_xticklabels(xlabel)
74
75 ax.set_xlabel("Q path")
76 ax.set_ylabel("Phonons [cm-1]")
77
78 ax.legend()
79
80 plt.tight_layout()
81 plt.savefig("dispersion.png")
82 plt.show()

```

If we save the script as `plot_dispersion.py` in the same directory as the calculation, we can run it with:

```
1 $ python plot_dispersion.py
```

Comparison between the SSCHA and the harmonic phonon dispersion of Gold.

Figure 1.4: Comparison between the SSCHA and the harmonic phonon dispersion of Gold.

The script will plot the figure of the phonon dispersion shown above. It is quite different from the experimental one because of the poor accuracy of the force field; however, the SSCHA result is much closer to the

experimental value.

Exercise:

Try to perform the simulation of Gold at a different temperature, then plot the SSCHA phonon dispersion as a function of temperature.

How do the phonon bands behave if the temperature is increased? Do they become more rigid (energy increases) or softer?

1.3 Running in the NPT Ensemble: Simulating Thermal Expansion

Now that you have some experience with the NVT simulation, we are ready for the next step: NPT, or relaxing the lattice.

With python-sscha it is very easy to run NPT simulation. You simply have to replace the line of the NVT script with the target pressure for the simulation:

```
1 # Replace the line
2 # relax.relax(get_stress = True)
3 # with
4 relax.vc_relax(target_press = 0)
```

And that is all! The target pressure is expressed in GPa; in this case, 0 is ambient conditions (1 atm = 0.0001 GPa).

You can also perform NVT simulation with variable lattice parameters. In this case the system will constrain the total volume to remain constant, but the lattice parameters will be optimized (if the system is not cubic and has some degrees of freedom, which is not the case for Gold).

The NVT ensemble with variable lattice parameters (cell shape) is:

```
1 # Replace the line
2 #     relax.vc_relax(target_press = 0)
3 # with
4 relax.vc_relax(fix_volume = True)
```

Indeed, this is an NVT simulation, so there is no need to specify the target pressure.

In the following script, we run the NPT ensemble at various temperatures, each time starting from the previous ensemble, to follow the volume thermal expansion of gold. This script assumes you have already performed the NVT calculation, so we can start from that result and avoid the harmonic calculation (it is always a good practice to start with an NVT simulation and then run NPT from the final result).

```
1 # Import the sscha code
2 import sscha, sscha.Ensemble, sscha.SchaMinimizer, sscha.Relax
3 import sscha.Utilities
4
5 # Import the cellconstructor library to manage phonons
6 import cellconstructor as CC, cellconstructor.Phonons
7 import cellconstructor.Structure, cellconstructor.calculators
8
9 # Import the force field of Gold
10 import ase, ase.calculators
11 from ase.calculators.emt import EMT
12
13 # Import numerical and general purpose libraries
14 import numpy as np, matplotlib.pyplot as plt
```

```

15 import sys, os
16
17
18 # Define the temperature range (in K)
19 T_START = 300
20 T_END = 1000
21 DT = 50
22
23 N_CONFIGS = 50
24 MAX_ITERATIONS = 10
25
26 # Import the gold force field
27 calculator = EMT()
28
29 # Import the starting dynamical matrix (final result of get_gold_free_energy.py)
30 dyn = CC.Phonons.Phonons("sscha_T300_dyn", nqirr = 13)
31
32 # Create the directory on which to store the output
33 DIRECTORY = "thermal_expansion"
34 if not os.path.exists(DIRECTORY):
35     os.makedirs("thermal_expansion")
36
37 # We cycle over several temperatures
38 t = T_START
39
40
41 volumes = []
42 temperatures = []
43 while t <= T_END:
44     # Change the temperature
45     ensemble = sscha.Ensemble.Ensemble(dyn, t)
46     minim = sscha.SchaMinimizer.SSCHA_Minimizer(ensemble)
47     minim.set_minimization_step(0.1)
48
49     relax = sscha.Relax.SSCHA(minim, calculator, N_configs = N_CONFIGS,
50                               max_pop = MAX_ITERATIONS)
51
52     # Setup the I/O
53     ioinfo = sscha.Utilities.IOInfo()
54     ioinfo.SetupSaving( os.path.join(DIRECTORY, "minim_t{}".format(t)))
55     relax.setup_custom_functions( custom_function_post = ioinfo.CFP_SaveAll)
56
57
58     # Run the NPT simulation
59     relax.vc_relax(target_press = 0)
60
61     # Save the volume and temperature
62     volumes.append(relax.minim.dyn.structure.get_volume())
63     temperatures.append(t)
64
65     # Start the next simulation from the converged value at this temperature
66     relax.minim.dyn.save_qe(os.path.join(DIRECTORY,
67                                           "sscha_T{}_dyn".format(t)))
68     dyn = relax.minim.dyn
69
70     # Print in standard output
71     relax.minim.finalize()
72
73     # Update the temperature

```

```

74     t += DT
75
76     # Save the thermal expansion
77     np.savetxt(os.path.join(DIRECTORY, "thermal_expansion.dat"),
78               np.transpose([temperatures, volumes]),
79               header = "Temperature [K]; Volume [A^3]")

```

You can run the script as always with:

```

1 $ python thermal_expansion.py

```

And... done!

This calculation is going to require a bit more time, as we run multiple SSCHA calculations at several temperatures. After it finishes, you can plot the results written in the file `thermal_expansion/thermal_expansion.dat`.

A simple script to plot the thermal expansion (and fit the volumetric thermal expansion coefficient) is the following:

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 import scipy, scipy.optimize
5
6
7 # Load all the dynamical matrices and compute volume
8 DIRECTORY = "thermal_expansion"
9 FILE = os.path.join(DIRECTORY, "thermal_expansion.dat")
10
11 # Load the data from the final data file
12 temperatures, volumes = np.loadtxt(FILE, unpack = True)
13
14
15 # Prepare the figure and plot the V(T) from the sscha data
16 plt.figure(dpi = 150)
17 plt.scatter(temperatures, volumes, label = "SSCHA data")
18
19 # Fit the data to estimate the volumetric thermal expansion coefficient
20 def parabola(x, a, b, c):
21     return a + b*x + c*x**2
22 def diff_parab(x, a, b, c):
23     return b + 2*c*x
24
25 popt, pcov = scipy.optimize.curve_fit(parabola, temperatures, volumes,
26                                     p0 = [0,0,0])
27
28 # Evaluate the volume thermal expansion
29 vol_thermal_expansion = diff_parab(300, *popt) / parabola(300, *popt)
30 plt.text(0.6, 0.2, r"$\alpha_v = "+ "{:.1f}".format(vol_thermal_expansion*1e6)+r"\times"
31         r"10^6 $ K$^{-1}$",
32         transform = plt.gca().transAxes)
33
34 # Plot the fit
35 _t_ = np.linspace(np.min(temperatures), np.max(temperatures), 1000)
36 plt.plot(_t_, parabola(_t_, *popt), label = "Fit")
37
38 # Adjust the plot adding labels, legend, and saving in eps
39 plt.xlabel("Temperature [K]")
40 plt.ylabel(r"Volume [Å^3]")

```

```

41 plt.legend()
42 plt.tight_layout()
43 plt.savefig("thermal_expansion.png")
44 plt.show()

```

Thermal expansion of Gold.

Figure 1.5: Thermal expansion of Gold.

We report the final thermal expansion in the figure above. The volumetric expansion coefficient α_v is obtained from the fit thanks to the thermodynamic relation:

$$\alpha_v = \frac{1}{V} \left(\frac{dV}{dT} \right)_P$$

Also in this case, the result is quite off from experiments, due to the not completely realistic force field employed. To get a more realistic result, you should use ab initio calculations or a more refined force field.

1.4 Ab Initio Calculation with the SSCHA Code

The SSCHA code is compatible with the Atomic Simulation Environment (ASE), which we used in the previous tutorial to get a fast force field for Gold.

However, ASE already provides an interface with most codes to run ab initio simulations. The simplest way of interfacing the SSCHA to another ab initio code is to directly use ASE.

The only difference is in the definition of the calculator. In the first example of this chapter, the Gold force field was defined as:

```

1 import ase
2 from ase.calculators.emt import EMT
3 calculator = EMT()

```

We simply need to replace these lines with our favorite DFT code. In this example we are going to use Quantum ESPRESSO, but the procedure for VASP, CASTEP, CRYSTAL, ABINIT, SIESTA, or your favorite one is exactly the same (refer to the official documentation of ASE for instructions on how to initialize these calculators).

In the case of DFT, unfortunately, we cannot simply create the calculator in one line, like we did for the EMT force field, as we also need to provide many parameters, such as pseudopotentials, the choice of exchange-correlation functional, the cutoff of the basis set, and the k-mesh grid for Brillouin zone sampling.

In the following example, we initialize the Quantum ESPRESSO calculator for Gold.

```

1 import cellconstructor.calculators
2 from ase.calculators.espresso import Espresso
3
4 # Initialize the DFT (Quantum Espresso) calculator for gold
5 # The input data is a dictionary that encodes the pw.x input file namelist
6 input_data = {
7     'control' : {
8         # Avoid writing wavefunctions on the disk
9         'disk_io' : 'None',
10        # Where to find the pseudopotential
11        'pseudo_dir' : '.',
12        'tprnfor' : True,    # Print the forces in output
13        'tstress' : True    # Print the stress tensor

```

```

14     },
15     'system' : {
16         # Specify the basis set cutoffs
17         'ecutwfc' : 45, # Cutoff for wavefunction
18         'ecutrho' : 45*4, # Cutoff for the density
19         # Information about smearing (it is a metal)
20         'occupations' : 'smearing',
21         'smearing' : 'mv',
22         'degauss' : 0.03
23     },
24     'electrons' : {
25         'conv_thr' : 1e-8
26     }
27 }
28
29 # the pseudopotential for each chemical element
30 # In this case just Gold
31 pseudopotentials = {'Au' : 'Au_ONCV_PBE-1.0.oncvpsp.upf'}
32
33 # the kpoints mesh and the offset
34 kpts = (1,1,1)
35 koffset = (1,1,1)
36
37
38 # Prepare the quantum espresso calculator
39 calculator = Espresso(input_data = input_data, pseudopotentials = pseudopotentials,
40                       kpts = kpts, koffset = koffset)

```

If you are familiar with the Quantum ESPRESSO input files, you should recognize all the options inside the `input_data` dictionary. For more options and more information, refer to the [Quantum ESPRESSO pw.x input guide](#).

Remember, the parameters set here are just for demonstration; remember to run appropriate convergence checks for the k-mesh, smearing, and basis set cutoffs before running the SSCHA code. Keep also in mind that this input file refers to the supercell, and the `kpts` variable should be properly rescaled if the supercell size is increased.

All the rest of the code remains the same (but here we do not compute harmonic phonons, which can be done more efficiently within Quantum ESPRESSO). Instead, we take the result obtained with EMT in the previous sections and try to relax the free energy with a fully ab initio approach.

The complete code is inside `Examples/sscha_and_dft/nvt_local.py`.

```

1  # Import the sscha code
2  import sscha, sscha.Ensemble, sscha.SchaMinimizer, sscha.Relax, sscha.Utilities
3
4  # Import the cellconstructor library to manage phonons
5  import cellconstructor as CC, cellconstructor.Phonons
6  import cellconstructor.Structure, cellconstructor.calculators
7
8  # Import the DFT calculator
9  import cellconstructor.calculators
10 from ase.calculators.espresso import Espresso
11
12 # Import numerical and general purpose libraries
13 import numpy as np, matplotlib.pyplot as plt
14 import sys, os
15
16
17 # Initialize the DFT (Quantum Espresso) calculator for gold

```

```

18 # The input data is a dictionary that encodes the pw.x input file namelist
19 input_data = {
20     'control' : {
21         # Avoid writing wavefunctions on the disk
22         'disk_io' : 'None',
23         # Where to find the pseudopotential
24         'pseudo_dir' : '.'
25     },
26     'system' : {
27         # Specify the basis set cutoffs
28         'ecutwfc' : 45, # Cutoff for wavefunction
29         'ecutrho' : 45*4, # Cutoff for the density
30         # Information about smearing (it is a metal)
31         'occupations' : 'smearing',
32         'smearing' : 'mv',
33         'degauss' : 0.03
34     },
35     'electrons' : {
36         'conv_thr' : 1e-8
37     }
38 }
39
40 # the pseudopotential for each chemical element
41 # In this case just Gold
42 pseudopotentials = {'Au' : 'Au_ONCV_PBE-1.0.oncvpsp.upf'}
43
44 # the kpoints mesh and the offset
45 kpts = (1,1,1)
46 koffset = (1,1,1)
47
48 # Specify the command to call quantum espresso
49 command = 'pw.x -i PREFIX.pwi > PREFIX.pwo'
50
51
52 # Prepare the quantum espresso calculator
53 calculator = Espresso(input_data = input_data,
54                       pseudopotentials = pseudopotentials,
55                       command = command,
56                       kpts = kpts,
57                       koffset = koffset)
58
59
60 TEMPERATURE = 300
61 N_CONFIGS = 50
62 MAX_ITERATIONS = 20
63 START_DYN = 'harmonic_dyn'
64 NQIRR = 13
65
66 # Let us load the starting dynamical matrix
67 gold_dyn = CC.Phonons.Phonons(START_DYN, NQIRR)
68
69 # Initialize the random ionic ensemble
70 ensemble = sscha.Ensemble.Ensemble(gold_dyn, TEMPERATURE)
71
72 # Initialize the free energy minimizer
73 minim = sscha.SchaMinimizer.SSCHA_Minimizer(ensemble)
74 minim.set_minimization_step(0.01)
75
76 # Initialize the NVT simulation

```



```

77 relax = sscha.Relax.SSCHA(minim, calculator, N_configs = N_CONFIGS,
78                           max_pop = MAX_ITERATIONS)
79
80 # Define the I/O operations
81 # To save info about the free energy minimization after each step
82 ioinfo = sscha.Utilities.IOInfo()
83 ioinfo.SetupSaving("minim_info")
84 relax.setup_custom_functions(custom_function_post = ioinfo.CFP_SaveAll)
85
86
87 # Run the NVT simulation (save the stress to compute the pressure)
88 relax.relax(get_stress = True)
89
90 # If instead you want to run a NPT simulation, use
91 # The target pressure is given in GPa.
92 #relax.vc_relax(target_press = 0)
93
94 # You can also run a mixed simulation (NVT) but with variable lattice parameters
95 #relax.vc_relax(fix_volume = True)
96
97 # Now we can save the final dynamical matrix
98 # And print in stdout the info about the minimization
99 relax.minim.finalize()
100 relax.minim.dyn.save_qe("sscha_T{}_dyn".format(TEMPERATURE))

```

Now you can run the SSCHA with an ab initio code! However, your calculation will probably take forever. Let us discuss parallelization and how to exploit modern HPC infrastructures to speed things up.

1.5 Parallelization

If you actually tried to run the code from the previous section on a laptop, it would take forever. The reason is that DFT calculations are much more expensive than the SSCHA minimization. While the SSCHA minimizes the number of ab initio calculations (especially when compared with MD or PIMD), they remain the bottleneck of the computational time.

For this reason, we need an appropriate parallelization strategy to reduce the total time to run an SSCHA.

The simplest way is to call the previous Python script with MPI:

```

1 $ mpirun -np 50 python nvt_local.py > output.log

```

The code will split the configurations in each ensemble across different MPI processes. In this case, we have 50 configurations per ensemble; by splitting them into 50 processors, we run the full ensemble in parallel.

However, even a single DFT calculation on 1 processor is going to take hours, and in some cases it may even take days. Luckily, Quantum ESPRESSO (and many other software) also has internal parallelization to work with. For example, we can tell Quantum ESPRESSO to run itself in parallel on 8 processors. For this purpose, we simply need to modify the command used to run Quantum ESPRESSO in the previous script.

```

1 # Lets replace
2 # command = 'pw.x -i PREFIX.pwi > PREFIX.pwo'
3 # with
4 command = 'mpirun -np 8 pw.x -npool 1 -i PREFIX.pwi > PREFIX.pwo'
5
6 # The command string is passed to the espresso calculator
7 calculator = CC.calculators.Espresso(input_data,
8                                     pseudopotentials,
9                                     command = command,

```

```
10         kpts = kpts,  
11         koffset = koffset)
```

In this way, our calculations will run on 400 processors (50 processors split the ensemble times 8 processors per calculation). This is achieved by nesting MPI calls. However, only the CellConstructor calculators can nest MPI calls without raising errors. This is the reason why we imported the Espresso class from CellConstructor and not from ASE. If you want to use ASE for your calculator, you can only use the internal parallelization of the calculator by modifying the command, as ASE itself implements an MPI parallelization on I/O operations that conflicts with the python-sscha parallelization. This limitation only applies to `FileIOCalculators` from ASE (so the EMT force field is not affected and can be safely used with python-sscha parallelization).

With this setup, the full code is parallelized over 400 processors. However, the SSCHA minimization algorithm is serial, and all the time spent in the actual SSCHA minimization is wasting the large number of allocated resources. Moreover, the SSCHA code needs to be configured and correctly installed on the cluster, which may be a difficult operation due to the hybrid Fortran/Python structure.

Chapter 2

Hands-on Session 3 - Calculations of Second-Order Phase Transitions with the SSCHA

In this hands-on session, we learn how to calculate second-order phase transitions within the SSCHA.

2.1 Structural Instability: Calculation of the Hessian

According to Landau's theory, a second-order phase transition occurs when the curvature of the free energy around a high-symmetry structure becomes negative along the direction of the order parameter that leads to a lower free energy.

Landau's theory of second-order phase transitions.

Figure 2.1: Landau's theory of second-order phase transitions.

For structural phase transitions, the order parameter is associated with atomic displacements, i.e. with phonon modes. Therefore, the curvature of the free energy is directly related to the eigenvalues of the free energy Hessian:

$$\frac{\partial^2 F}{\partial R_a \partial R_b}$$

Here, $F(R)$ denotes the free energy as a function of the average atomic positions R , which correspond to the centroids of the atomic probability distribution in the SSCHA.

Thus, the free energy Hessian is the key quantity for investigating structural instabilities and phase transitions. Within the SSCHA framework, it can be computed using an analytical expression derived in [Bianco et al., Phys. Rev. B 96, 014111](#):

$$\frac{\partial^2 F}{\partial R_a \partial R_b} = \Phi_{ab} + \sum_{cdef} \Phi_{acd}^{(3)} [1 - \Lambda \Phi_{cdef}^{(4)}]^{-1} \Phi_{efb}^{(3)}$$

In practice, given an ensemble of configurations, this quantity can be evaluated stochastically through the function call:

```
ensemble.get_free_energy_hessian()
```

By diagonalizing the free energy Hessian, its eigenvalues can be tracked as a function of external parameters such as temperature and pressure, allowing one to investigate structural phase transitions and identify the onset of instabilities. The corresponding eigenvectors describe the atomic displacement patterns (distortion modes) associated with these instabilities.

We now show how to compute the free energy Hessian starting from an SSCHA minimization. As a practical example, we consider CsPbI₃ and compute the Hessian of the high-symmetry phase as a function of temperature T .

For each temperature T , the starting point is the set of SSCHA dynamical matrices dyn_q obtained from the SSCHA minimization for that temperature.

As done in the previous tutorial, energies and forces are computed using a GAP (Gaussian Approximation Potential) force fields within the quippy framework. Starting from the SSCHA dynamical matrix, an ensemble of atomic configurations is generated at temperature T . The corresponding energies and forces are then evaluated using the GAP calculator.

We begin by importing the required modules:

```
1 import numpy as np
2
3 import cellconstructor as CC
4 import cellconstructor.Phonons
5
6 import sscha
7 import sscha.Ensemble
8
9 from quippy.potential import Potential
```

We then define the main parameters of the calculation:

```
1 # Temperature in Kelvin
2 T = 0
3
4 # Number of configurations in the ensemble
5 N = 100
6
7 # Prefix of the SSCHA dynamical matrices
8 dyn_prefix = "dyn"
9
10 # Number of irreducible q-points
11 nqirr = 4
12
13 # GAP file
14 gap_file = "GAP.xml"
```

We load the GAP calculator:

```
1 calculator = Potential("IP GAP", param_filename=gap_file)
```

We load the SSCHA dynamical matrices:

```
1 dyn = CC.Phonons.Phonons(dyn_prefix, nqirr)
```

Starting from these dynamical matrices, we generate an ensemble and compute energies and forces with the GAP calculator:

```
1 ensemble = sscha.Ensemble.Ensemble(dyn, T)
2 ensemble.generate(N)
3 ensemble.compute_ensemble(calculator)
```

We now compute the free energy Hessian matrix:

```
1 ensemble.update_weights(dyn, T)
2 hessian = ensemble.get_free_energy_hessian(include_v4=False)
```

Here, `include_v4=False` neglects higher-order four-phonon scattering terms in the SSCHA phonon self-energy. This approximation makes the calculation lighter, as four-phonon processes require a large memory allocation for big systems, scaling as $(3N)^4$ with N the number of atoms in the supercell. Moreover, they may require a larger number of configurations to achieve convergence. However, this approximation is not only sufficient for a first analysis, but it often provides results that are quantitatively reliable for many systems. For this reason, in this tutorial we neglect the fourth-order contribution.

Finally, we save the Hessian in Quantum ESPRESSO dynamical-matrix format:

```
1 hessian.save_qe("hessian_")
```

We can diagonalize the Hessian to inspect its eigenvalues and eigenvectors:

```
1 w_hessian, pols_hessian = hessian.DiagonalizeSupercell()
2
3 frequencies = w_hessian * CC.Units.RY_TO_CM
4
5 print("\n".join(["{:16.4f} cm-1".format(w) for w in frequencies]))
```

The eigenvalues of the free energy Hessian determine the local stability of the structure. Positive frequencies indicate stable modes, while imaginary frequencies indicate structural instabilities. The corresponding eigenvectors describe the atomic displacement patterns associated with the unstable modes.

2.2 Temperature Dependence of the Free Energy Hessian

We now analyze how the free energy Hessian evolves as a function of temperature.

Starting from a reference SSCHA dynamical matrix dyn_q , we compute the free energy Hessian at different temperatures without performing a full SSCHA minimization at each step. This allows us to efficiently investigate the stability of the high-symmetry phase.

We define a range of temperatures:

```
1 import numpy as np
2
3 temperatures = np.linspace(100, 600, 6)
```

We store the lowest eigenvalues of both the SSCHA dynamical matrix and the free energy Hessian:

```
1 lowest_hessian_mode = []
```

We then loop over the temperatures:

```
1 for T in temperatures:
2
3     # Generate ensemble
4     ensemble = sscha.Ensemble.Ensemble(dyn, T)
5     ensemble.generate(N)
6
7     # Compute energies and forces using GAP
8     ensemble.compute_ensemble(calculator)
9
10    # Compute free energy Hessian
```

```

11 hessian = ensemble.get_free_energy_hessian(include_v4=False)
12
13 # Diagonalize Hessian
14 w_hessian, pols_hessian = hessian.DiagonalizeSupercell()
15
16 # Remove acoustic modes
17 acoustic_modes = CC.Methods.get_translations(
18     pols_hessian,
19     dyn.structure.generate_supercell(dyn.GetSupercell()).get_masses_array()
20 )
21
22 w_hessian = w_hessian[~acoustic_modes]
23
24 # Store lowest mode
25 lowest_hessian_mode.append(np.min(w_hessian) * CC.Units.RY_TO_CM)

```

We save the results:

```

1 data = np.zeros((len(temperatures), 2))
2 data[:, 0] = temperatures
3 data[:, 1] = lowest_hessian_mode
4
5 np.savetxt(
6     "hessian_vs_temperature.dat",
7     data,
8     header="T [K]; Free energy Hessian [cm-1]"
9 )

```

2.3 Analysis

We can now plot the temperature dependence of the lowest Hessian mode:

```

1 import matplotlib.pyplot as plt
2
3 data = np.loadtxt("hessian_vs_temperature.dat")
4
5 plt.figure(dpi=120)
6 plt.plot(data[:,0], data[:,1], marker="o")
7
8 plt.axhline(0, color="k", ls="dotted")
9 plt.xlabel("Temperature [K]")
10 plt.ylabel("Frequency [cm-1]")
11
12 plt.tight_layout()
13 plt.savefig("hessian_vs_temperature.png")

```

2.4 Interpretation

The lowest eigenvalue of the free energy Hessian provides direct information about the stability of the structure:

- Positive values → stable phase

- Negative values $\rightarrow$ structural instability

By tracking this quantity as a function of temperature, it is possible to identify the onset of instabilities and characterize structural phase transitions.

2.5 Exercise

- Increase the number of configurations N and analyze the convergence of the Hessian.
- Compare results obtained with and without `include_v4`.
- Identify the distortion pattern associated with the lowest eigenmode.

```

1  #!/usr/bin/env python
2  # -*- coding: utf-8 -*-
3  #
4  #  SSCHA_exercise_Calculus.py
5  #
6  # Import the cellconstructor stuff
7  import cellconstructor as CC
8  import cellconstructor.Phonons
9  import cellconstructor.ForceTensor
10 import cellconstructor.Structure
11 import cellconstructor.Spectral
12
13 # Import the modules of the force field
14 import fforces as ff
15 import fforces.Calculator
16
17 # Import the modules to run the sscha
18 import sscha, sscha.Ensemble, sscha.SchaMinimizer
19 import sscha.Relax, sscha.Utilities
20
21 import spglib
22 from ase.visualize import view
23
24 # Import Matplotlib to plot
25 import numpy as np
26 import matplotlib.pyplot as plt
27 from matplotlib import cm
28 import timeit

```

Next, we set some variables for the calculation:

```

1  #Setting the variables:
2  #Setting the temperature in Kelvin:
3  Temperature = 0
4  #Setting the number of configurations:
5  configurations = 50
6  #Setting the names and location of the files:
7  Files_dyn_SnTe = "ffield_dynq"
8  #Set the number of irreducible q (related to the supercell size):
9  nqirr = 3
10 #Setting the frequencies output file:
11 File_frequencies = "frequencies.dat"
12 #Setting the dynamical matrix output filename:
13 File_final_dyn = "final_sscha_T{ }_".format(int(Temperature))

```

Now we need to calculate the SSCHA dynamical matrix. For that, we follow several steps:

1. First, we prepare the Toy model force field that substitutes the usual ab initio calculations for this tutorial. This force field needs the harmonic dynamical matrix to be initialized, along with the higher-order parameters. Finally, the dynamical matrix for the minimization is loaded and ready. Since we are studying a system that has a spontaneous symmetry breaking at low temperature, the harmonic dynamical matrices will have imaginary phonons. We must enforce phonons to be positive definite to start an SSCHA minimization.

```

1  # Load the dynamical matrix for the force field
2  ff_dyn = CC.Phonons.Phonons("ffield_dynq", 3)
3
4  # Setup the forcefield with the correct parameters
5  ff_calculator = ff.Calculator.ToyModelCalculator(ff_dyn)
6  ff_calculator.type_cal = "pbtex"
7  ff_calculator.p3 = 0.036475
8  ff_calculator.p4 = -0.022
9  ff_calculator.p4x = -0.014
10
11 # Initialization of the SSCHA matrix
12 dyn_sscha = CC.Phonons.Phonons(Files_dyn_SnTe, nqirr)
13 # Flip the imaginary frequencies into real ones
14 dyn_sscha.ForcePositiveDefinite()
15 # Apply the ASR and the symmetry group
16 dyn_sscha.Symmetrize()

```

2. The next step is to create the ensemble for the specified temperature. As an extra, we also look for the space group of the structure.

```

1  ensemble = sscha.Ensemble.Ensemble(dyn_sscha,
2      T0 = Temperature, supercell = dyn_sscha.GetSupercell())
3  # Detect space group
4  symm=spglib.get_spacegroup(dyn_sscha.structure.get_ase_atoms(),
5      0.005)
6  print('Initial SG = ', symm)

```

As before, we begin by importing some libraries and setting variables:

```

1  #!/usr/bin/env python
2  # -*- coding: utf-8 -*-
3  #
4  # SSCHA_exercise_Unstable.py
5  #
6  # Import the cellconstructor stuff
7  import cellconstructor as CC
8  import cellconstructor.Phonons
9  import cellconstructor.ForceTensor
10 import cellconstructor.Structure
11 import cellconstructor.Spectral
12
13 # Import the modules of the force field
14 import fforces as ff
15 import fforces.Calculator
16
17 # Import the modules to run the sscha
18 import sscha, sscha.Ensemble, sscha.SchaMinimizer
19 import sscha.Relax, sscha.Utilities
20
21 import spglib
22 from ase.visualize import view
23

```



```

24 # Import Matplotlib to plot
25 import numpy as np
26 import matplotlib.pyplot as plt
27 from matplotlib import cm
28 import timeit
29
30 #Setting the variables:
31 #Setting the temperature in Kelvin:
32 Temperature = 0
33 #Setting the number of configurations:
34 configurations = 50
35 #Setting the names and location of the files:
36 Files_dyn_SnTe = "ffield_dynq"
37 #Set the number of irreducible q (related to the supercell size):
38 nqirr = 3
39 #Setting the frequencies output file:
40 File_frequencies = "frequencies.dat"
41 #Setting the dynamical matrix output filename:
42 File_final_dyn = "final_sscha_T{}_".format(int(Temperature))

```

Now we look for that instability:

1. The ff_calculator toy potential is defined as we have seen in the previous program.

```

1 # Load the dynamical matrix for the force field
2 ff_dyn = CC.Phonons.Phonons("ffield_dynq", 3)
3
4 # Setup the forcefield with the correct parameters
5 ff_calculator = ff.Calculator.ToyModelCalculator(ff_dyn)
6 ff_calculator.type_cal = "pbtex"
7 ff_calculator.p3 = 0.036475
8 ff_calculator.p4 = -0.022
9 ff_calculator.p4x = -0.014
10
11 # Initialization of the SSCHA matrix
12 dyn_sscha = CC.Phonons.Phonons(Files_dyn_SnTe, nqirr)
13 dyn_sscha.ForcePositiveDefinite()
14
15 # Apply also the ASR and the symmetry group
16 dyn_sscha.Symmetrize()

```

2. Next, we load the dynamical matrix calculated previously with the ff_calculator toy potential, so there is no need to calculate it again.

```

1 # The SSCHA dynamical matrix is needed (the one after convergence)
2 # We reload the final result (no need to rerun the sscha minimization)
3 dyn_sscha_final = CC.Phonons.Phonons(File_final_dyn, nqirr)

```

3. Then, since the Hessian calculation is more sensitive, we generate a new ensemble with more configurations. To compute the Hessian we will use an ensemble of 10000 configurations. Note that we can use fewer if we use the Sobol sequence, or we can load a previously generated ensemble.

```

1 # We reset the ensemble
2 ensemble = sscha.Ensemble.Ensemble(dyn_sscha_final, T0 = Temperature,
3                                     supercell = dyn_sscha_final.GetSupercell())
4
5 # We need a bigger ensemble to properly compute the hessian
6 # Here we will use 10000 configurations
7 ensemble.generate(5000, sobol = True)

```

```

8  #ensemble.generate(10000, sobol = False)
9  #We could also load the ensemble with
10 # ensemble.load("data_ensemble_final", N = 100, population = 5)

```

4. We now compute forces and energies using the force field calculator.

```

1  # We now compute forces and energies using the force field calculator
2  ensemble.get_energy_forces(ff_calculator, compute_stress = False)

```

5. Finally, the free energy Hessian is calculated with the `get_free_energy_hessian` function. We can choose whether to neglect or include the four-phonon scattering process. Four-phonon scattering processes require a huge memory allocation for big systems, scaling as $(3N)^4$ with N the number of atoms in the supercell. Moreover, they may also require more configurations to converge.

In almost all the systems we have studied up to now, we found this four-phonon scattering at high order to be negligible. We remark that the SSCHA minimization already includes four-phonon scattering at the lowest order perturbation theory; thus, neglecting this term only affects combinations of one or more four-phonon scattering with two three-phonon scatterings (high-order diagrams). For more details, see [Bianco et al., Phys. Rev. B 96, 014111](#).

We can then print the frequencies of the Hessian. If an imaginary frequency is present, then the system wants to spontaneously break the high-symmetry phase.

```

1  print("Updating the importance sampling...")
2  # If the sscha matrix was not the one used to compute the ensemble
3  # We must update the ensemble weights
4  # We can also use this function to simulate a different temperature.
5  ensemble.update_weights(dyn_sscha_final, Temperature)
6  # ----- COMPUTE THE FREE ENERGY HESSIAN -----
7  print("Computing the free energy hessian...")
8  dyn_hessian = ensemble.get_free_energy_hessian(include_v4 = False)
9  # We neglect high-order four phonon scattering
10 #dyn_hessian = ensemble.get_free_energy_hessian(include_v4 = True,
11 #        get_full_hessian = True, verbose = True) # Full calculus
12 # We can save the free energy hessian as a dynamical matrix
13 # in quantum espresso format
14 dyn_hessian.save_qe("hessian")
15 # -----
16 # We calculate the frequencies of the hessian:
17 w_hessian, pols_hessian = dyn_hessian.DiagonalizeSupercell()
18
19 # Print all the frequency converting them into cm-1 (They are in Ry)
20 print("\n".join(["{:16.4f} cm-1".format(w * CC.Units.RY_TO_CM) for w in w_hessian]))

```

The frequencies in the free energy Hessian are temperature dependent.

We can look at the eigenmodes of the free energy Hessian to check if we have imaginary phonons. If there are negative frequencies, then we have found an instability. You can check what happens if you include the fourth order.

Exercise:

The Sobol sequences reduce the number of configurations by providing a better mapping of the Gaussian than a random distribution. By uniformly spreading the samples with a low-discrepancy sequence like Sobol, it is possible to reduce the number of configurations needed. Low-discrepancy sequences tend to sample space “more uniformly” than random numbers. Algorithms that use such sequences may have superior convergence. You can test this in the calculation of the Hessian by changing the number of configurations and the mapping scheme in the `ensemble.generate()` function.

2.6 Second Order Phase Transition

Up to now, we studied the system at $T = 0$ K and found that there is an instability. However, we can repeat the minimization at many temperatures and track the phonon frequency to see at which temperature the system becomes stable.

Again, we load and set the variables. Now we have several temperatures, so we store them in an array:

```
1  #!/usr/bin/env python
2  # -*- coding: utf-8 -*-
3  #
4  #  SSCHA_exercise_Unstable.py
5  #
6  # Import the cellconstructor stuff
7  import cellconstructor as CC
8  import cellconstructor.Phonons
9  import cellconstructor.ForceTensor
10 import cellconstructor.Structure
11 import cellconstructor.Spectral
12
13 # Import the modules of the force field
14 import fforces as ff
15 import fforces.Calculator
16
17 # Import the modules to run the sscha
18 import sscha, sscha.Ensemble, sscha.SchaMinimizer
19 import sscha.Relax, sscha.Utilities
20
21 import spglib
22 from ase.visualize import view
23
24 # Import Matplotlib to plot
25 import numpy as np
26 import matplotlib.pyplot as plt
27 from matplotlib import cm
28 import timeit
29
30 #Setting the variables:
31 #Setting the temperature in Kelvin:
32 Temperature = 0
33 #Setting the number of configurations:
34 configurations = 50
35 #Setting the names and location of the files:
36 Files_dyn_SnTe = "ffield_dynq"
37 #Set the number of irreducible q (related to the supercell size):
38 nqirr = 3
39 #Setting the frequencies output file:
40 File_frequencies = "frequencies.dat"
41 #Setting the dynamical matrix output filename:
```

```

42 File_final_dyn = "final_sscha_T{}_".format(int(Temperature))
43 sobol = False
44 sobol_scatter = False

```

1. Like in the previous program, first we prepare the Toy model force field:

```

1 # Load the dynamical matrix for the force field
2 ff_dyn = CC.Phonons.Phonons("ffield_dynq", 3)
3
4 # Setup the forcefield with the correct parameters
5 ff_calculator = ff.Calculator.ToyModelCalculator(ff_dyn)
6 ff_calculator.type_cal = "pbtex"
7 ff_calculator.p3 = 0.036475
8 ff_calculator.p4 = -0.022
9 ff_calculator.p4x = -0.014

```

2. We are going to need a range of temperatures for this calculation:

```

1 # Define the temperatures, from 50 to 300 K, 6 temperatures
2 temperatures = np.linspace(50, 300, 6)
3
4 lowest_hessian_mode = []
5 lowest_sscha_mode = []
6
7 # Perform a simulation at each temperature
8 t_old = Temperature

```

3. In the next part, we condense the calculation of the Hessians in a loop over different temperatures. At the end, it searches for the lowest non-acoustic frequency and saves it together with the corresponding auxiliary SSCHA frequency.

```

1 for Temperature in temperatures:
2     # Load the starting dynamical matrix
3     dyn = CC.Phonons.Phonons(File_final_dyn.format(int(t_old)), nqirr)
4
5     # Prepare the ensemble
6     ensemble = sscha.Ensemble.Ensemble(dyn, T0 = Temperature,
7                                         supercell = dyn.GetSupercell())
8
9     # Prepare the minimizer
10    minim = sscha.SchaMinimizer.SSCHA_Minimizer(ensemble)
11    minim.min_step_struc = 0.05
12    minim.min_step_dyn = 0.002
13    minim.kong_liu_ratio = 0.5
14    minim.meaningful_factor = 0.000001
15    #minim.root_representation = "root4"
16    #minim.precond_dyn = False
17    #minim.minim_struct = True
18    #minim.neglect_symmetries = True
19    minim.enforce_sum_rule = True # Lorenzo's solution to the error
20
21    # Prepare the relaxer (through many population)
22    relax = sscha.Relax.SSCHA(minim, ase_calculator = ff_calculator,
23                              N_configs=configurations, max_pop=20)
24
25    # Relax
26    relax.relax(sobol = sobol, sobol_scramble = sobol_scatter)
27    #relax.relax()
28

```

```

29     # Save the dynamical matrix
30     relax.minim.dyn.save_qe(File_final_dyn.format(int(Temperature)))
31
32     # Detect space group
33     symm=spglib.get_spacegroup(relax.minim.dyn.structure.get_ase_atoms(),
34                               0.005)
35     print('Current SG = ', symm, ' at T=',int(Temperature))
36
37     # Recompute the ensemble for the hessian calculation
38     ensemble = sscha.Ensemble.Ensemble(relax.minim.dyn, T0 = Temperature,
39                                       supercell = dyn.GetSupercell())
40     ensemble.generate(configurations, sobol = sobol,
41                     sobol_scramble = sobol_scatter)
42     ensemble.get_energy_forces(ff_calculator, compute_stress = False)
43     #gets the energies and forces from ff_calculator
44
45     #update weights!!!
46     ensemble.update_weights(relax.minim.dyn, Temperature)
47     # Get the free energy hessian
48     dyn_hessian = ensemble.get_free_energy_hessian(include_v4 = False)
49     #free energy hessian as in Bianco paper 2017
50     dyn_hessian.save_qe("hessian_T{}_".format(int(Temperature)))
51
52     # Get the lowest frequencies for the sscha and the free energy hessian
53     w_sscha, pols_sscha = relax.minim.dyn.DiagonalizeSupercell() #dynamical matrix
54     # Get the structure in the supercell
55     superstructure = relax.minim.dyn.structure.generate_supercell(relax.minim.dyn.
56                           GetSupercell())
57
58     # Discard the acoustic modes
59     acoustic_modes = CC.Methods.get_translations(pols_sscha,
60                                                  superstructure.get_masses_array())
61     w_sscha = w_sscha[~acoustic_modes]
62
63     lowest_sscha_mode.append(np.min(w_sscha) * CC.Units.RY_TO_CM) # Convert from Ry
64     to cm-1
65
66     w_hessian, pols_hessian = dyn_hessian.DiagonalizeSupercell() #recomputed dyn
67     for hessian
68     # Discard the acoustic modes
69     acoustic_modes = CC.Methods.get_translations(pols_hessian,
70                                                  superstructure.get_masses_array())
71     w_hessian = w_hessian[~acoustic_modes]
72     lowest_hessian_mode.append(np.min(w_hessian) * CC.Units.RY_TO_CM) # Convert
73     from Ry to cm-1
74     #print ("\n".join(["{:.4f} cm-1".format(w * CC.Units.RY_TO_CM) for w in
75     pols_hessian]))
76     #exit()
77
78     t_old = Temperature
79
80     # We prepare now the file to save the results
81     freq_data = np.zeros( (len(temperatures), 3))
82     freq_data[:, 0] = temperatures
83     freq_data[:, 1] = lowest_sscha_mode
84     freq_data[:, 2] = lowest_hessian_mode
85
86     # Save results on file
87     np.savetxt("{}_hessian_vs_temperature.dat".format(configurations),
88               freq_data, header = "T [K]; SSCHA mode [cm-1]; Free energy hessian [cm

```

```
-1] ")
```

4. Finally, we produce a graphical output of the data.

```
1 hessian_data = np.loadtxt("{}_hessian_vs_temperature.dat".format(configurations))
2
3 plt.figure(dpi = 120)
4 plt.plot(hessian_data[:,0], hessian_data[:,1],
5          label = "Min SCHA freq", marker = ">")
6 plt.plot(hessian_data[:,0], hessian_data[:,2],
7          label = "Free energy curvature", marker = "o")
8 plt.axhline(0, 0, 1, color = "k", ls = "dotted") # Draw the zero
9 plt.xlabel("Temperature [K]")
10 plt.ylabel("Frequency [cm-1]")
11 plt.legend()
12 plt.tight_layout()
13 plt.savefig('{} _Temp_Freq.png'.format(configurations))
14 plt.show()
15
16 plt.figure(dpi = 120)
17 plt.plot(hessian_data[:,0], np.sign(hessian_data[:,2]) * hessian_data[:,2]**2,
18          label = "Free energy curvature", marker = "o")
19 plt.axhline(0, 0, 1, color = "k", ls = "dotted") # Draw the zero
20 plt.xlabel("Temperature [K]")
21 plt.ylabel("$\omega^2$ [cm-2]")
22 plt.legend()
23 plt.tight_layout()
24 plt.savefig('{} _Temp_Omeg.png'.format(configurations))
25 plt.show()
```

We will simulate temperatures up to room temperature (300 K) with steps of 50 K. Note that this will perform all the steps above 6 times, so it may take some minutes depending on the PC (on an i3 from 2015, with one core, it took 2 hours). If it takes too long, you can reduce the number of steps by changing the temperature array in `Temperature_i = np.linspace(50, 300, 6)`.

Frequencies versus Temperatures

Figure 2.2: Frequencies versus Temperatures

In the figure above, we can see that the phase transition is between 100 K and 150 K. The data points do not form a linear figure. We can increase the number of temperature points to locate the exact transition temperature, but there is another better way to find it.

Squared frequencies versus Temperatures

Figure 2.3: Squared frequencies versus Temperatures

For the Landau theory of phase transitions, since the SSCHA is a mean-field approach, we expect that around the transition the critical exponent of the temperature goes as:

$$\omega \sim \sqrt{\Phi}$$

For this reason, it is better to plot the temperature versus the square of the frequency, as in the figure above. This makes the graph linear, so we can easily estimate the critical temperature by linear interpolation.

We are using only 50 configurations in the ensemble. Note that this makes the calculation fast, but it is a low number for these calculations because the free energy calculations are noisier than the SSCHA frequencies.

This is because the computation of the free energy requires the third-order force constant tensor, which requires more configurations to converge.

Exercise:

How does the calculation of the free energy change with the number of configurations?

Evolution of the lowest *soft* frequency in relation to the number of configurations in the ensemble with a stable configuration. The line is the mean and the shade is the standard deviation.

Figure 2.4: Evolution of the lowest *soft* frequency in relation to the number of configurations in the ensemble with a stable configuration. The line is the mean and the shade is the standard deviation.

Exercise:

Plot the Hessian phonon dispersion.

Hessian phonon dispersion

Figure 2.5: Hessian phonon dispersion

Workflow of the SSCHA objects for: A) Free energy minimization; B) Structural instabilities search; C) Temperature loop for second order phase transition code. Dotted lines are functions within objects. The dotted and dashed lines indicate the relationship of the dynamic matrix to the objects.

Figure 2.6: Workflow of the SSCHA objects for: A) Free energy minimization; B) Structural instabilities search; C) Temperature loop for second order phase transition code. Dotted lines are functions within objects. The dotted and dashed lines indicate the relationship of the dynamic matrix to the objects.

Chapter 3

Hands-on Session 3 - Spectral functions, Raman and infrared spectra with the Time-Dependent SSCHA

3.1 Theoretical background

This tutorial focuses on the computation of dynamical properties of materials. The key quantity which is measured by experiments is the *dynamical response function* $\chi(\omega)$. The response function probes how the material responds to a time-dependent external perturbation. We can model the result of any experiment as: the material is in equilibrium for $t < t_0$. Then, at t_0 a new perturbation is turned on, and we measure a property A of the material at a certain time t . This response will be the convolution of the perturbation at all times between t_0 and t , weighted by how the perturbation propagates in time, which is precisely the response function $\chi(t - t')$:

$$A(t) = A_0 + \int_{t_0}^t dt' \chi(t - t') F(t')$$

where $F(t')$ is the external time-dependent perturbation, and A_0 is the value of the property A in equilibrium. Alternatively, by passing in Fourier space, the convolution becomes a simple product:

$$A(\omega) = A_0 + \chi(\omega) F(\omega)$$

This is very general, and it applies to any experiment. In this tutorial, we focus on IR and Raman spectroscopy. For the case of IR, the perturbation is the electric field of the light, which interacts with the material by generating an oscillating dipole. The response of the material is the *emitted* (or absorbed) electric field, which is related to how the dipole moment of the material oscillates in time. So the observable $A(t)$ is the dipole moment of the material, and the perturbation $F(t)$ is the amplitude of the electric field of the light.

The Kubo equation for the response function is

$$\chi(t) = \frac{i}{\hbar} \theta(t) \langle M(t) M(0) \rangle$$

where $\hat{M}(t)$ is the dipole moment operator in the Heisenberg picture, and $\langle \cdot \rangle$ is the quantum average at finite temperature. The SSCHA can compute the response function $\chi(t)$ by using the TD-SCHA formalism. Here we will use the Lanczos algorithm to compute the response function in Fourier space, which is more efficient than computing it in time and then Fourier transforming it.

The first step is to convert the diopole operator in phonons creation and annihilation operators. To this aim, we can approximate the dipole moment as a linear function of the atomic displacements $\hat{u}$:

$$\hat{M}(t) = \left. \frac{\partial M}{\partial u} \right|_{u=0} \hat{u}(t) + O(u^2)$$

The derivative of the dipole moment with respect to the atomic displacements is precisely the Born effective charge:

$$Z_{\alpha\beta}^i = \frac{\partial M_\alpha}{\partial u_\beta^i} = \frac{\partial^2 \mathcal{E}}{\partial u_\beta^i \partial E_\alpha}$$

where E_α is the electric field in the α Cartesian direction, u_β^i is the displacement of atom i in the β Cartesian direction, and $\mathcal{E}$ is the total energy of the system (Born-Oppenheimer). Using this in the expression of the susceptibility, we obtain:

$$\chi_{\text{IR}\alpha}(t) = \sum_{ij} \sum_{\beta\gamma} \frac{Z_{\alpha\beta}^i Z_{\alpha\gamma}^j}{\sqrt{m_i m_j}} \sqrt{m_i m_j} \langle \hat{u}_\beta^i(t) \hat{u}_\gamma^j(0) \rangle$$

The last term is the so-called phonon Green's function, which represent the propagation of a phonon created at time $t = 0$ and destroyed at time t .

$$G_{\alpha\beta}^{ij}(t) = \sqrt{m_i m_j} \langle \hat{u}_\alpha^i(t) \hat{u}_\beta^j(0) \rangle$$

Analogously, the Raman spectrum is related to the response of the material to two electric fields, which interact with the material by generating an oscillating polarizability. The observable $A(t)$ is the polarizability of the material, and the perturbation $F(t)$ is the amplitude of the two overlapping electric fields which interfere within the material. The Raman susceptibility can be written as:

$$\chi_{\text{Raman}}(t) = \langle \hat{\alpha}_{\alpha\beta}(t) \hat{\alpha}_{\alpha\beta}(0) \rangle$$

where $\hat{\alpha}_{\alpha\beta}$ is the polarizability operator, which can be approximated as a linear function of the atomic displacements:

$$\hat{\alpha}_{\alpha\beta}(t) = \left. \frac{\partial \alpha_{\alpha\beta}}{\partial u_\gamma^i} \right|_{u=0} \hat{u}_\gamma^i(t) + O(u^2)$$

$$\Xi_{\alpha\beta\gamma}^i = \frac{\partial \alpha_{\alpha\beta}}{\partial u_\gamma^i} = \frac{\partial^3 \mathcal{E}}{\partial u_\gamma^i \partial E_\alpha \partial E_\beta}$$

where $\Xi_{\alpha\beta\gamma}^i$ is the Raman tensor, which is the third derivative of the total energy (Born-Oppenheimer) with respect to the atomic displacements and the two electric fields (incoming-outgoing).

Notably, the Raman requires two electric fields because it is a scattering, where incoming and outgoing radiation are different. The IR instead is an absorption/emission, where incoming and outgoing radiation are the same.

The Raman susceptibility is then:

$$\chi_{\text{Raman}}(t) = \langle \hat{\alpha}_{\alpha\beta}(t) \hat{\alpha}_{\alpha\beta}(0) \rangle = \sum_{ij} \sum_{\gamma\delta} \frac{\Xi_{\alpha\beta\gamma}^i \Xi_{\alpha\beta\delta}^j}{\sqrt{m_i m_j}} \sqrt{m_i m_j} \langle \hat{u}_\gamma^i(t) \hat{u}_\delta^j(0) \rangle$$

Also here, the last term is the phonon Green's function, which represent the propagation of a phonon created at time $t = 0$ and destroyed at time t . Therefore, to compute the IR and Raman spectra, we need to compute the phonon Green's function. Going in Fourier space, we get

$$\chi_{\text{IR}\alpha}(\omega) = \sum_{ij} \sum_{\beta\gamma} \frac{Z_{\alpha\beta}^i Z_{\alpha\gamma}^j}{\sqrt{m_i m_j}} \sqrt{m_i m_j} G_{\beta\gamma}^{ij}(\omega)$$

$$\chi_{\text{Raman}\alpha\beta}(\omega) = \sum_{ij} \sum_{\gamma\delta} \frac{\Xi_{\alpha\beta\gamma}^i \Xi_{\alpha\beta\delta}^j}{\sqrt{m_i m_j}} \sqrt{m_i m_j} G_{\gamma\delta}^{ij}(\omega)$$

3.1.1 The phonon Green's function

The phonon Green's function can be computed within the TD-SCHA. In general, we can write the Green's function as a non-interacting green function plus a self-energy:

$$G^{-1}(\omega) = G_0^{-1}(\omega) - \Pi(\omega)$$

where $G_0(\omega)$ is a non-interacting Green's function for *harmonic*-like phonons, while $\Pi(\omega)$ is the phonon self-energy, which depends on the frequency. In the TD-SCHA theory, the non-interacting Green's function is defined as the Green's function of the auxiliary SSCHA harmonic Hamiltonian.

$$G_0^{-1}(\omega) = I\omega^2 - D_{\text{SSCHA}}$$

where I is the identity matrix and D_{SSCHA} is the dynamical matrix of the SSCHA Hamiltonian.

The self-energy $\Pi(\omega)$ correspond to the same expression derived for the Free energy Hessian; see [Bianco et al., Physical Review B, 96, 014111, 2017](#).

$$\Pi_{ab}(\omega) = -\frac{1}{2} \sum_{cd\mu\nu} \Phi_{acd}^{(3)} \Lambda_{\mu\nu}^{cd}(\omega) \left[1 + \frac{1}{2} \Phi \Lambda(\omega) \right]^{-1} \Phi_{\nu b}^{(3)}$$

where $\Lambda_{\mu\nu}^{cd}(\omega)$ is the Fourier transform of the two-phonon propagator, which can be computed from the equilibrium non interacting Green's function $G_0(\omega)$ as:

$$\Lambda_{\alpha\beta\gamma\delta}^{ijlm}(t) = \sqrt{m_i m_j m_l m_m} \langle \hat{u}_\alpha^i(t) \hat{u}_\beta^j(t) \hat{u}_\gamma^l(0) \hat{u}_\delta^m(0) \rangle_0$$

Computing the full inversion of the self-energy for every value of the frequency is computationally extremely expensive. It is possible, with an efficient algorithm (Lanczos), to prove that the Green function can be obtained by inverting a special matrix, see [Monacelli, Mauri, Physical Review B 103, 104305, 2021](#).

In this tutorial, we will use this Lanczos algorithm to compute the phonon Green's function and, consequently, the IR spectrum of CsPbI3.

For this, we need the package `tdscha` (it is recommended to configure it with the Julia speedup to run faster; see the installation guide).

3.2 Tutorial

We need to first relax a complete SSCHA calculation, exactly as for the free energy hessian. This can be performed via the `sscha_relax.py` script, which perform an automatic (fixed volume) sscha relaxation from the Harmonic dynamical matrix using 256 configurations.

We provide the final result already as `sscha_auxiliary_dyn_` files. The first step for a dynamical linear response calculation is to have a very well converged auxiliary dynamical matrix. For this purpose, it is useful to run an additional minimization with a higher number of configuration once.

This is done by the script `last_sscha_minim.py` which we will analyze in the next section.

```

1 import sys, os
2 import cellconstructor as CC, cellconstructor.Phonons
3 import sscha, sscha.Ensemble, sscha.SchaMinimizer
4
5 from quippy.potential import Potential
6
7
8 TEMPERATURE = 450 # K
9 NQIRR = 4

```

```

10 START_DYN = "sscha_auxiliary_dyn_"
11 POTENTIAL = "../Materials/GAP_1.xml"
12 N_CONFIGS = 1024
13 POP_ID = 100
14
15
16 def last_sscha_relax(temperature = TEMPERATURE):
17     # Load the sscha dynamical matrix
18     dyn = CC.Phonons.Phonons(START_DYN, NQIRR)
19
20     # Load the interatomic Potential for CsPbI3
21     calc = Potential("IP GAP", param_filename=POTENTIAL)
22
23     # Generate the last ensemble
24     ensemble = sscha.Ensemble.Ensemble(dyn, temperature)
25     ensemble.generate(N_CONFIGS)
26
27     # Compute the energies and forces of the ensemble with the GAP potential
28     ensemble.compute_ensemble(calc)
29
30     # Compute a full sscha minimization on the new bigger ensemble
31     minim = sscha.SchaMinimizer.SSCHA_Minimizer(ensemble)
32     minim.set_minimization_step(0.02)
33     minim.meaningful_factor = 0.01
34     minim.run()
35
36     # Save the final dynamical matrix and ensemble for further calculations
37     minim.ensemble.save_bin("data", POP_ID)
38     minim.dyn.save_qe("sscha_converged_dyn_")
39
40
41 last_sscha_relax()

```

This script performs a full SSCHA minimization starting from the auxiliary dynamical matrix obtained from the previous SSCHA relaxation, but using a bigger ensemble of 1024 configurations. The final ensemble is saved in binary format inside the directory `data`, with a unique ID of 100 (any other ensemble with that ID will be overwritten). The final SSCHA auxiliary dynamical matrix is saved in the file `sscha_converged_dyn_`. These are useful if you want to perform multiple linear response calculations, as they do not need to be recomputed.

Once we have the final converged auxiliary dynamical matrix, we can compute the phonon Green's function and the IR spectrum with the script `tdscha_run_ir.py`.

```

1 import cellconstructor as CC, cellconstructor.Phonons
2 import sscha, sscha.Ensemble
3 import numpy as np
4
5 import tdscha, tdscha.DynamicalLanczos
6
7 import sys, os
8
9 TEMPERATURE = 450 # K
10 NQIRR = 4 # Irreducible q points of the dynamical matrix
11
12 # Info about the dynamical matrix and the ensemble
13 ORIGINAL_DYN = "sscha_auxiliary_dyn_"
14 FINAL_DYN = "sscha_converged_dyn_"
15 POP_ID = 100
16

```

```

17 def compute_ir():
18     # Load the original ensemble
19     dyn_original = CC.Phonons.Phonons(ORIGINAL_DYN, NQIRR)
20     ensemble = sscha.Ensemble.Ensemble(dyn_original, TEMPERATURE)
21     ensemble.load_bin("data", POP_ID)
22
23     # Lets load the final converged dynamical matrix
24     final_dyn = CC.Phonons.Phonons(FINAL_DYN, NQIRR)
25
26     # To prepare the IR or Raman, we need
27     # IR : dielectric tensor and Born effective charges
28     # Raman : Raman tensor
29     # Load them from quantum espresso ph.x output
30     final_dyn.ReadInfoFromESPRESSO("dielectric_calc.pho")
31
32     # Update the ensemble weights on the converged dynamical matrix
33     ensemble.update_weights(final_dyn, TEMPERATURE)
34
35     # Initialize the TD-SCHA Lanczos algorithm
36     lanczos = tdscha.DynamicalLanczos.Lanczos(ensemble)
37     lanczos.init()
38
39     # Let us define which level of anharmonicity we want
40     lanczos.ignore_v3 = False # Add bubble contribution if false
41     lanczos.ignore_v4 = True # Add RPA resummation if false (a factor 2 slower in speed
42                             # - no extra memory)
43
44     # If both v3 and v4 are ignored (both true), we get the 'harmonic' spectrum
45     # on the sscha auxiliary frequencies
46
47     # Define the reponse function to observe
48     # In this case IR with polarization along the x axis
49     polarization = np.array([1.0, 0.0, 0.0])
50     lanczos.prepare_ir(pol_vec = polarization)
51
52     # Run the lanczos algorithm for 40 steps
53     lanczos.run_FT(40)
54
55     # Save the final result in binary
56     lanczos.save_status("IR_x.npz")
57
58 compute_ir()

```

3.2.1 Deep analysis of the script

Loading the ensemble

Let's analyze the script `tdscha_run_ir.py` step by step inside the function `compute_ir()`: The first step is loading the final sscha ensemble and dynamical matrix, which is achieved by the following lines:

```

1     dyn_original = CC.Phonons.Phonons(ORIGINAL_DYN, NQIRR)
2     ensemble = sscha.Ensemble.Ensemble(dyn_original, TEMPERATURE)
3     ensemble.load_bin("data", POP_ID)
4
5     # Lets load the final converged dynamical matrix
6     final_dyn = CC.Phonons.Phonons(FINAL_DYN, NQIRR)

```

Born effective charges and Raman tensors

Notably, we want to compute the IR response. For this reason, we need to tell the `tdscha` the relation between atomic displacements and the polarization. This is achieved by loading the Born effective charges and dielectric tensor from a Quantum ESPRESSO phonon calculation, which is done by the line:

```
1 final_dyn.ReadInfoFromESPRESSO("dielectric_calc.pho")
```

The file `dielectric_calc.pho` is the output of a Quantum ESPRESSO phonon calculation, which contains the information about the Born effective charges and dielectric tensor. We provide the example files used to generate this output as `dielectric_calc.pwi` and `dielectric_calc.phi`. In particular, we need to extract the structure from the final dynamical matrix to run the Quantum ESPRESSO calculation to write the file `dielectric_calc.pwi`, as the Born effective charges needs to be computed on the final converged centroids positions. This is performed by the file `extract_structure.py`, in particular by the lines:

```
1 # Load the final converged dynamical matrix
2 dyn = CC.Phonons.Phonons("sscha_converged_dyn_")
3
4 # Save the structure in the espresso format for pw.x
5 dyn.structure.save_scf("sscha_structure.scf")
```

The final structure looks like the following

```
1 CELL_PARAMETERS angstrom
2 6.2340501132548267 0.0000000000000000 0.0000000000000000
3 0.0000000000000000 6.2340501132548267 0.0000000000000000
4 0.0000000000000000 0.0000000000000000 6.2340501132548267
5
6 ATOMIC_POSITIONS angstrom
7 Cs 3.1170250566274138 3.1170250566274138 3.1170250566274138
8 Pb 0.00000000000000002 0.00000000000000002 0.00000000000000002
9 I 3.1170250566274138 0.00000000000000002 0.00000000000000002
10 I 0.00000000000000002 0.00000000000000002 3.1170250566274138
11 I 0.00000000000000002 3.1170250566274138 0.00000000000000002
```

This file contains the cell parameters (rows of the cell matrix) in Angstrom and the cartesian coordinates of all the atoms in the primitive cell (also in Angstrom). In this format, it can be pasted on the bottom of a Quantum ESPRESSO input file.

The two espresso inputs can be run with the following commands (**No need to do it now, it may take time, we already provide the final output files**):

```
1 mpirun -np 4 pw.x -npool 4 -i dielectric_calc.pwi > dielectric_calc.pwo
2 mpirun -np 4 ph.x -npool 4 -i dielectric_calc.phi > dielectric_calc.pho
```

Once the effective charges, Raman Tensor and Dielectric Tensor have been computed, they can be loaded in the final dynamical matrix with the method `ReadInfoFromESPRESSO`, which is used in the line:

```
1 final_dyn.ReadInfoFromESPRESSO("dielectric_calc.pho")
```

This is important if we want to compute IR or Raman response, as the response functions are computed with the effective charges and Raman tensor. We can skip this if, instead, we want just the displacement-displacement green-function (e.g. to compute the free energy Hessian).

Update the ensemble

Next, we update the ensemble so that it reflects the final dynamical matrix, which is done by the line:

```
1 ensemble.update_weights(final_dyn, TEMPERATURE)
```

This is the reweighting, and it changes the weight of each stochastic configuration inside the ensemble so that, when we compute the average, we are computing the average with respect to the `final_dyn` rather than the dynamical matrix used to generate the ensemble. The new weights are computed as:

$$\rho_i = \frac{\rho_{\mathcal{R}_1, \Phi_1}(\mathbf{R}_i)}{\rho_{\mathcal{R}^{(0)}, \Phi^{(0)}}}$$

Initialize the Lanczos algorithm for dynamical linear response

Next, we need to prepare the grounds for the dynamical linear-response calculation. This is performed using the Lanczos algorithm. The algorithm is initialized as follows:

```

1  # Initialize the TD-SCHA Lanczos algorithm
2  lanczos = tdscha.DynamicalLanczos.Lanczos(ensemble)
3  lanczos.init()
4
5  # Let us define which level of anharmonicity we want
6  lanczos.ignore_v3 = False # Add bubble contribution if false
7  lanczos.ignore_v4 = True # Add RPA resummation if false

```

The `ignore_v3` and `ignore_v4` flags determine if we compute the phonon spectra with different approximation. By setting both to False, we are computing the full TD-SCHA response function, without any other approximation. Setting `ignore_v4` to True is equivalent to setting `include_v4` to False in the Free energy Hessian. However, thanks to how the Lanczos algorithm is formulated, including the full RPA resummation with the 4-phonon scattering vertices does not increase the computational cost of the algorithm. In particular, the cost only increase by a factor of 2, without any consequences on the memory. Here, we ignore it just to have quick results, however, you can try running it also with False. This is also the reason why we can use the Lanczos algorithm from the TD-SCHA to compute the free energy Hessian accounting for the 4-phonon scatterings even in relatively large supercells without any memory issue.

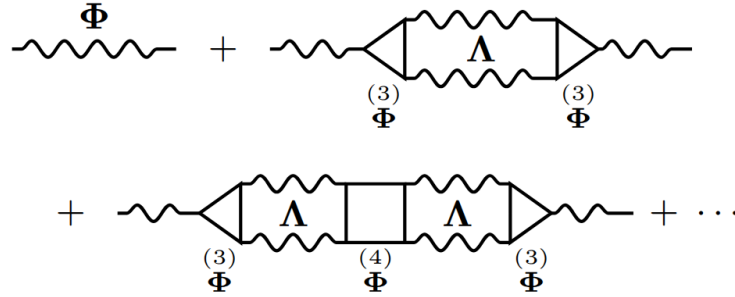


Figure 3.1: Diagrammatic series of perturbation present in the self-energy. The first term is the bare auxiliary harmonic propagation, the 3-phonon scattering vertex $\Phi^{(3)}$ enable the bubble diagram (the second term) while the 4-phonon scattering $\Phi^{(4)}$ enables all the chained diagrams obtained from the RPA like Dyson equation.

Define the perturbation

Then, we need to specify which response function we want to compute. In this case IR response, with polarization along the x cartesian axis:

```

1  # Define the reponse function to observe
2  # In this case IR with polarization along the x axis
3  polarization = np.array([1.0, 0.0, 0.0])
4  lanczos.prepare_ir(pol_vec = polarization)

```

This call needs to know which atom and how they displace when we perturb the system with an electric field on the x axis. This information is contained in the effective charges, which are read from the `dielectric_calc.pho` file, therefore this call will crash if no effective charge is defined on the dynamical matrix. If we want to run the Raman, we would have use the `prepare_raman` function instead, which needs the Raman tensor instead of the effective charges. Indeed, for the raman, we need to pass both the input and output polarization vector for the electric field (scattering). If we wanted the displacement-displacement Green function, there is no need to preload the effective charges or the Raman tensor. In this case, we just need to call `prepare_mode` passing the index of the phonon mode we want to perturb. The index of the phonon mode is ordered from 0 (the lowest frequency mode in the supercell) to $3N-3$ (the highest frequency mode in the supercell), excluding the 3 acoustic modes at Gamma.

Run the algorithm

Last, we can run the Lanczos algorithm.

```
1 # Run the lanczos algorithm for 40 steps
2 lanczos.run_FT(40)
```

The Lanczos algorithm is iterative. We get to decide how many iterations. Feel free to reduce this to 20-30 iterations if it takes too much time. Each iteration computes the next Lanczos vector and the next element of the tridiagonal matrix, which is used to compute the response function. The more iterations we do, the better the resolution of the spectra. The run command takes multiples optional arguments. Let us see the API documentation for the `run_FT` method to see what we can do:

```
1 run_FT(n_iter, save_dir = None, save_each = 5, verbose = True, n_rep_ortho = 0,
        n_ortho = 10, flush_output = True, debug = False, prefix = "LANCZOS", run_simm
        = None, optimized = False)
```

Among all these option, worth mention is `save_dir`, `save_each` and `prefix` keywords. The `save_dir`, if set to something different to `None`, is the directory in which the intermediate status of the Lanczos algorithm is saved. This can be used to resume a previous calculation, or to check the convergence of the spectra with the number of iterations before we reached the maximum number of iterations. `save_each` is the number of iterations after which the status is saved, and `prefix` is the prefix of the filename in which the status is saved. By default, the status is not saved, you can try to set `save_dir` to something like `'.'` to see that every 5 steps it will save a file with the name `LANCZOS_step_XX.npz`, where `XX` is the number of iterations.

We can also manually save at the end of the run with the method `save_status` of the Lanczos object, which takes as argument the filename in which the status is saved.

```
1 # Save the final result in binary
2 lanczos.save_status("IR_x.npz")
```

If we run the algorithm, we will get the final spectra in the file `IR_x.npz`.

3.2.2 Extract the green's function and plot

The TD-SCHA provides automatically a very simple way to plot the spectra with the command-line utility `tdscha-plot-data`

```
1 tdscha-plot-data IR_x.npz 0 200 0.5
```

The first argument is the file in which the lanczos calculation is performed, then we give a range of frequencies (in cm^{-1}), here from 0cm^{-1} to 200cm^{-1} , the last one is the smearing (in cm^{-1}) used to plot the spectra. This command-line provides a quick way to plot the spectra. However, if you want more control, we can get the spectrum directly in python.

The example script that plots the spectrum is `plot_spectrum.py`.

```

1 import cellconstructor as CC, cellconstructor.Units
2 import tdscha, tdscha.DynamicalLanczos
3 import numpy as np
4 import matplotlib.pyplot as plt
5 import sys, os
6
7 DATA = "IR_x.npz"
8 W_START = 0 # cm-1
9 W_END = 200 # cm-1
10 TERMINATOR = True
11 SMEARING = 0.5 # cm-1
12
13 def plot_spectrum():
14     # Load the final lanczos results
15     lanczos = tdscha.DynamicalLanczos.Lanczos()
16     lanczos.load_status(DATA)
17
18     # Extract the Green function
19     w_array = np.linspace(W_START, W_END, 2000)
20     w_ry = w_array / CC.Units.RY_TO_CM
21     smearing_ry = SMEARING / CC.Units.RY_TO_CM
22     green_function = lanczos.get_green_function_continued_fraction(w_ry, smearing=
23                                     smearing_ry,
24                                     use_terminator =
25                                     TERMINATOR)
26
27     # The IR spectrum is proportional to - Im (G(w))
28     ir_spectrum = -np.imag(green_function)
29
30     # Plot the IR spectrum
31     plt.plot(w_array, ir_spectrum)
32     plt.xlabel("Frequency [cm-1]")
33     plt.ylabel("- Im (G)")
34     plt.title("IR spectrum - polarization x")
35     plt.tight_layout()
36     plt.savefig("ir_spectrum.png")
37     plt.show()
38 plot_spectrum()

```

The final result is plotted in the following figure

The continued fraction

In particular, we first load the Lanczos algorithm status:

```

1 # Load the final lanczos results
2 lanczos = tdscha.DynamicalLanczos.Lanczos()
3 lanczos.load_status("IR_x.npz")

```

We then need to compute the Green's function from the Lanczos coefficient. The Lanczos algorithm find an orthonormal basis in which the inverse-response function is a tridiagonal matrix $\mathcal{T}$

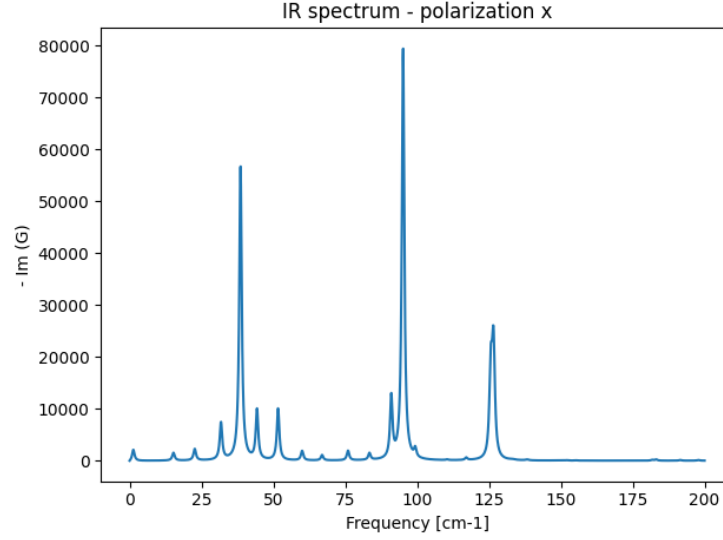


Figure 3.2: IR Spectrum obtained with the `plot_spectrum.py`.

$$\mathcal{T} = \begin{pmatrix} a_1 & b_1 & 0 & 0 & \dots \\ b_1 & a_2 & b_2 & 0 & \dots \\ 0 & b_2 & a_3 & b_3 & \dots \\ 0 & 0 & b_3 & a_4 & \dots \\ \vdots & \vdots & \vdots & \vdots & \ddots \end{pmatrix}$$

where the first element of the basis is the perturbation vector. Therefore, the green function is actually the (1,1) element of the inverse of the tridiagonal matrix:

$$G(\omega + i\eta) = [\mathcal{T} - \mathcal{I}(\omega + i\eta)^2]_{1,1}^{-1}$$

where $\mathcal{I}$ is the identity matrix. Thanks to the many zeros in the tridiagonal matrix, the inverse of the first element is very easy to compute, and correspond to the following continued fraction:

$$G(\omega + i\eta) = \frac{1}{a_1 - (\omega + i\eta)^2 - \frac{b_1^2}{a_2 - (\omega + i\eta)^2 - \frac{b_2^2}{a_3 - (\omega + i\eta)^2 - \dots}}}$$

The function of the python script `get_green_function_continued_fraction` uses the `a_i`, `b_i` saved inside the `lanczos` object to compute the green function in the frequencies provided in input:

```
1 green_function = lanczos.get_green_function_continued_fraction(w_ry, smearing=
2     smearing_ry,
                                     use_terminator =
                                     TERMINATOR)
```

This function takes as input the frequency ω (in Ry units), the smearing η , and if to use or not a *terminator*. The terminator is a trick to reach the $N \rightarrow \infty$ limit (where N is the number of iterations). Empirically, we see that after a certain number of iteration, the coefficients `a_i` and `b_i` oscillate around a specific value. Therefore we can fill all the values for $i > N$ with the average value of the coefficient, and simulate an infinite continued fraction. The infinite continued fraction can be solved analytically:

$$G_{\infty}(z) = \frac{1}{a_{\infty} - z^2 - b_{\infty}G_{\infty}(z)}$$

$$G_{\infty}(z) (a_{\infty} - z^2 - b_{\infty}G_{\infty}(z)) = 1$$

By solving this equation, we can replace the last fraction with the $G_{\infty}(z)$, simulating infinite iterations. Setting `use_terminator` to `True`

Exercise:

Compute the Lanczos with the SSCHA auxiliary frequencies, the bubble approximation and without any approximation, and check the differences.

Question:

Is this calculation enough, or do we need to compute also the response function for other perturbation, like y and z ? Will something change in the spectrum if we do? Why?

Exercise:

Plot the IR data at various smearings and as a function of the number of steps (10, 20, 30, 40). How does the signal change with smearing and the number of steps? Try plotting with and without the terminator and see the differences.

3.3 Raman Response

The Raman response is very similar to the IR. Raman probes the fluctuations of the polarizability instead of those of the polarization, and it occurs when the sample interacts with two light sources: the incoming electromagnetic radiation and the outgoing one. The outgoing radiation has a frequency that is shifted with respect to the incoming one by the energy of the scattering phonons. The signal on the red side of the pump is called Stokes, while the signal on the blue side is the Anti-Stokes. Since the outgoing radiation has higher energy than the incoming one in the Anti-Stokes, it is generated only by existing (thermally excited) phonons inside the sample, and therefore it has a lower intensity than the Stokes.

If we insert this into the expression for the intensity, the average between the positions is the atomic Green's function divided by the square root of the masses, giving:

$$I(\omega) \propto \sum_{ab} \frac{\Xi_{xya}\Xi_{xyb}}{\sqrt{m_a m_b}} [-\text{Im}G_{ab}(\omega)] (n(\omega) + 1)$$

where $G_{ab}(\omega)$ is the atomic Green's function on atoms a and b , while Ξ_{xya} is the Raman tensor along the electric fields directed in x and y on atom a .

The multiplication factor $n(\omega) + 1$ comes from the observation of the Stokes nonresonant Raman (it would be just $n(\omega)$ for the Anti-Stokes).

As we did for the IR signal, we can prepare the calculation of the Raman scattering by computing the polarizability-polarizability response function.

```

1 # Setup the polarized Raman response
2 polarization_in = np.array([1,0,0])
3 polarization_out = np.array([1,0,0])
4 lanczos.prepare_raman(pol_vec_in=polarization_in,
5                       pol_vec_out=polarization_out)

```

Note that here we have to specify two polarizations of the light: the incoming radiation and the outgoing radiation.

Indeed, we need the Raman tensor Ξ_{xya} defined within the dynamical matrix.

For the cubic phase, Ξ_{abc} is zero by symmetry, therefore there is no Raman active mode. To compute the Raman response, we need a lower symmetry phase.

In this case, we take the tetragonal (β) phase of CsSnI_3 . We provide inside the **Materials** directory a already performed SSCHA relaxation at 300 K (`sscha_auxiliary_tetra_files`)

Also in this case, we need to perform a new final relaxation. This final relaxation can be performed with the `last_sscha_minim_tetra.py`, which is very similar to the script we employed for the cubic phase

```

1 import cellconstructor as CC, cellconstructor.Phonons
2 import sscha, sscha.Ensemble, sscha.SchaMinimizer
3
4 from quippy.potential import Potential
5
6
7 TEMPERATURE = 300 # K
8 NQIRR = 3
9 START_DYN = "sscha_auxiliary_tetra_"
10 POTENTIAL = "../Materials/GAP_1.xml"
11 N_CONFIGS = 1024
12 POP_ID = 200
13
14
15 def last_sscha_relax(temperature = TEMPERATURE):
16     # Load the sscha dynamical matrix
17     dyn = CC.Phonons.Phonons(START_DYN, NQIRR)
18
19     # Load the interatomic Potential for CsPbI3
20     calc = Potential("IP GAP", param_filename=POTENTIAL)
21
22     # Generate the last esemble
23     ensemble = sscha.Ensemble.Ensemble(dyn, temperature)
24     ensemble.generate(N_CONFIGS)
25
26     # Compute the energies and forces with the GAP potential
27     ensemble.compute_ensemble(calc)
28
29     # Compute a full sscha minimization on the new bigger ensemble
30     minim = sscha.SchaMinimizer.SSCHA_Minimizer(ensemble)
31     minim.set_minimization_step(0.02)
32     minim.meaningful_factor = 0.01
33     minim.run()
34
35     # Save the final dynamical matrix and ensemble for further calculations
36     minim.ensemble.save_bin("data", POP_ID)
37     minim.dyn.save_qe("sscha_converged_tetra_dyn_")
38
39
40 last_sscha_relax()

```

This script is very similar to the previous one, the main differences are: 1. The number of irreducible q-points NQIRR, which is 3 instead of 4 (we have a 2x2x1 supercell, so less q points in total) 2. The temperature is set to 300 K instead of 450 K 3. The name of the dynamical matrix is obviously different 4. We save the ensemble in population 200 to avoid overwriting the cubic one in population 100.

Once this script runs, we get the new ensemble and the final highly converged dynamical matrix. At this point, we need to compute the Raman tensor, using DFPT (e.g. using quantum espresso). The Raman tensor express how the polarizability change when we move each atom. CellConstructor can load it from the quantum espresso ph.x output, like the Born effective charges.

The script to compute the raman is:

```

1 import cellconstructor as CC, cellconstructor.Phonons
2 import sscha, sscha.Ensemble
3 import numpy as np
4
5 import tdscha, tdscha.DynamicalLanczos
6
7 import sys, os
8
9 TEMPERATURE = 300 # K
10 NQIRR = 3 # Irreducible q points of the dynamical matrix
11
12 # Info about the dynamical matrix and the ensemble
13 ORIGINAL_DYN = "sscha_auxiliary_tetra_"
14 FINAL_DYN = "sscha_converged_tetra_dyn_"
15 POP_ID = 200
16
17 def compute_raman():
18     # Load the original ensemble
19     dyn_original = CC.Phonons.Phonons(ORIGINAL_DYN, NQIRR)
20     ensemble = sscha.Ensemble.Ensemble(dyn_original, TEMPERATURE)
21     ensemble.load_bin("data", POP_ID)
22
23     # Lets load the final converged dynamical matrix
24     final_dyn = CC.Phonons.Phonons(FINAL_DYN, NQIRR)
25
26     # To prepare the Raman, we need to load the Raman tensor
27     # Load them from quantum espresso ph.x output
28     final_dyn.ReadInfoFromESPRESSO("dielectric_calc_tetra.pho")
29
30     # Update the ensemble weights on the converged dynamical matrix
31     ensemble.update_weights(final_dyn, TEMPERATURE)
32
33     # Initialize the TD-SCHA Lanczos algorithm
34     lanczos = tdscha.DynamicalLanczos.Lanczos(ensemble, lo_to_split = None)
35     lanczos.init()
36
37     # Let us define which level of anharmonicity we want
38     lanczos.ignore_v3 = False # Add bubble contribution if false
39     lanczos.ignore_v4 = True # Add RPA resummation if false (a factor 2 slower in speed
40                             # - no extra memory)
41
42     # If both v3 and v4 are ignored (both true), we get the 'harmonic' spectrum
43     # on the sscha auxiliary frequencies
44
45     # Define the reponse function to observe
46     # In this case IR with polarization along the x axis
47     polarization_in = np.array([1.0, 0.0, 0.0])
48     polarization_out = np.array([1.0, 0.0, 0.0])
49     lanczos.prepare_raman(pol_vec_in = polarization_in,
50                          pol_vec_out = polarization_out
51                          )
52
53     # Run the lanczos algorithm for 40 steps

```

```

53     lanczos.run_FT(40)
54
55     # Save the final result in binary
56     lanczos.save_status("Raman_xx.npz")
57
58 if __name__ == "__main__":
59     # Change the working directory to the one containing this script
60     os.chdir(os.path.dirname(os.path.abspath(__file__)))
61     compute_raman()

```

Also this script is very similar to the IR one, with a few main differences:

1. We load the data of the tetragonal phase.
2. The raman initialization

Let us focus on the Raman initialization:

```

1     # Define the reponse function to observe
2     # In this case IR with polarization along the x axis
3     polarization_in = np.array([1.0, 0.0, 0.0])
4     polarization_out = np.array([1.0, 0.0, 0.0])
5     lanczos.prepare_raman(pol_vec_in = polarization_in,
6                           pol_vec_out = polarization_out)

```

In this case, we need an incoming field polarization and an outgoing field polarization. The simultaneous presence of incoming and outgoing radiation generates a force on the atoms that generate phonons inside the material.

The result is the following

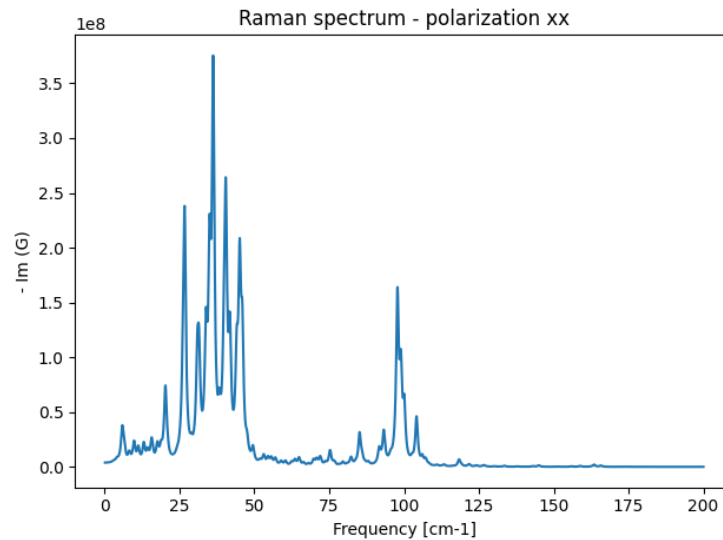


Figure 3.3: Raman spectrum with 200 iterations of the Lanczos algorithm along the x polarization for both incoming and the outgoing radiation

Exercise:

Try to run the Raman off-diagonal like x-y, or the zz. Are the photo-excited modes the same?

The solution for the unpolarized Raman is something like

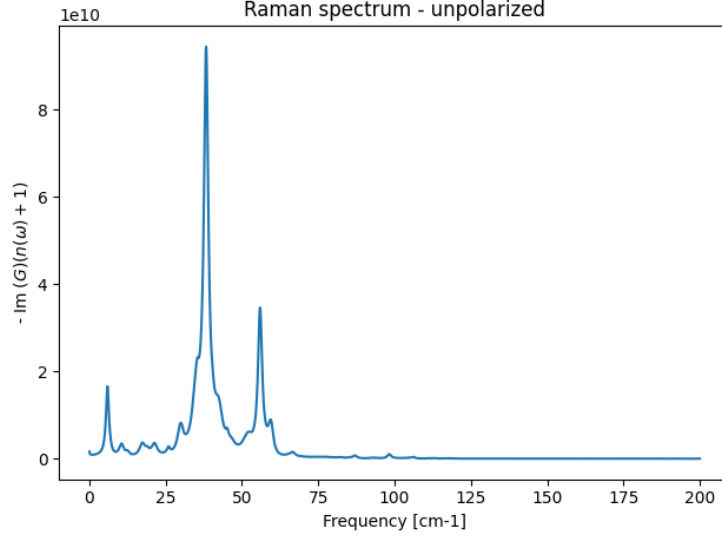


Figure 3.4: Unpolarized Raman. The scattering factor $n(\omega) + 1$ kills high frequency modes, which have a lower thermal population than low-frequency modes.

3.3.1 Unpolarized Raman

Often, experiment do not distinguish between laser polarization or sample orientation (e.g., the sample is a powder). In these cases, we need to average across all possible crystal orientation, or light polarization. This would require a Monte Carlo sampling, which could take a lot of computational time to compute the response function. An alternative approach is to directly compute the unpolarized Raman. In particular, the unpolarized Raman signal can be computed from the so-called *invariants*, where the perturbations in the polarizations are:

$$I_A = \frac{1}{9}(xx + yy + zz)^2$$

$$I_{B_1} = (xx - yy)^2/2$$

$$I_{B_2} = (xx - zz)^2/2$$

$$I_{B_3} = (yy - zz)^2/2$$

$$I_{B_4} = 3(xy)^2$$

$$I_{B_5} = 3(yz)^2$$

$$I_{B_6} = 3(xz)^2$$

The total intensity of unpolarized Raman is:

$$I_{unpol}(\omega) = 45 \cdot I_a(\omega) + 7 \cdot \sum_{i=1}^6 I_{B_i}(\omega)$$

The `tdscha` code implements a way to compute each perturbation separately. For example, the Raman response related to I_A is calculated with:

```
1 lanczos.prepare_raman(unpolarized=0)
```

While the I_{B_i} components are computed using index i . For example, to compute I_{B_5} :

```
1 # To compute I_B5 we do
2 lanczos.prepare_raman(unpolarized=5)
```

To obtain the total spectrum, you need to add the factor $n(\omega) + 1$ and sum all these perturbations with the correct prefactor (45 for I_A and 7 for the sum of all I_B). Note that the

To reset a calculation and start a new one, you can use:

```
1 lanczos.reset()
```

which may be called before preparing the perturbation.

This could be run inside a loop, e.g.

```
1 for i in range(7):
2     lanczos.prepare_raman(unpolarized=i)
3
4     # Run the lanczos algorithm for 40 steps
5     lanczos.run_FT(50)
6
7     # Save the final result in binary
8     lanczos.save_status(f"Raman_unpol{i}.npz")
9
10    # Reset the calculation, ready for a new perturbation
11    lanczos.reset()
```

Or, if the calculation is heavy, it could be run in parallel by simply running simultaneously the different perturbations.

Exercise:

Compute the unpolarized Raman spectrum of ice and plot the results.

Chapter 4

Hands-on Session 9: Thermal Conductivity Calculations with the SSCHA

In previous lessons, we saw how to calculate vibrational properties of materials using the stochastic self-consistent harmonic approximation (SSCHA). In this tutorial, we will use that knowledge to calculate the lattice thermal conductivity of materials.

To do this, we need the SSCHA dynamical matrices (**auxiliary dynamical matrices, not free-energy Hessians**) and the third-order force constants. These quantities allow us to calculate the harmonic properties, such as phonon frequencies and group velocities, and the anharmonic properties, such as phonon lifetimes and spectral functions. Together, these are the ingredients required for lattice thermal-conductivity calculations.

In this tutorial, we will:

- Use SSCHA dynamical matrices and third-order force constants to calculate lattice thermal conductivity with the Green-Kubo (GK) and single relaxation time approximation (SRTA) approaches.
- Learn how to converge lattice thermal-conductivity calculations.
- Manipulate `ThermalConductivity` objects to analyze the calculated transport properties.

The basic workflow is:

1. Relax the crystal structure and obtain SSCHA dynamical matrices in the temperature range of interest.
2. Use the relaxed SSCHA dynamical matrices to calculate the SSCHA free-energy Hessian, confirm dynamical stability, and obtain third-order force constants.
3. Calculate the lattice thermal conductivity and check its convergence with respect to:
 - the $\mathbf{q}$ -point grid used for the thermal-conductivity calculation;
 - the size of the SSCHA supercell;
 - the number of configurations used to calculate the third-order force constants;
 - the smearing parameter used in the phonon self-energy calculation, which is closely related to the size of the $\mathbf{q}$ -point grid when using an *adaptive* smearing scheme;
 - the density of frequency sampling controlled by the `ne` parameter when using the Green-Kubo method.

4.1 Lattice Thermal Conductivity of CsPbI₃

We first calculate the second- and third-order force constants.

```
1 import warnings
```



```

2
3 import ase
4 import ase.io
5 import numpy as np
6 from ase import Atoms
7 from ase.eos import calculate_eos
8 from ase.filters import ExpCellFilter
9 from ase.optimize import QuasiNewton
10 from ase.units import kJ
11 from quippy.potential import Potential
12
13 # Import the CellConstructor and SSCHA modules used throughout this tutorial.
14 import cellconstructor as CC
15 import cellconstructor.Phonons
16 import cellconstructor.Structure
17 import sscha
18 import sscha.Ensemble
19 import sscha.Relax
20 import sscha.SchaMinimizer
21
22 # NumPy moved ComplexWarning in newer versions. This block keeps the script
23 # compatible with both older and newer NumPy releases.
24 try:
25     ComplexWarning = np.exceptions.ComplexWarning
26 except AttributeError:
27     ComplexWarning = np.ComplexWarning
28
29 warnings.filterwarnings("ignore", category=ComplexWarning)
30
31
32 def get_starting_dynamical_matrices(structure_filename, potential, supercell):
33     """Relax the input structure and compute an initial dynamical matrix.
34
35     Parameters
36     -----
37     structure_filename : str
38         Path to the input structure in Quantum ESPRESSO format.
39     potential : quippy.potential.Potential
40         Interatomic potential used to compute energies and forces.
41     supercell : list[int]
42         Supercell used for the finite-displacement phonon calculation.
43
44     Returns
45     -----
46     dyn : cellconstructor.Phonons.Phonons
47         Initial auxiliary dynamical matrix.
48     bulk : float
49         Static bulk modulus, used during variable-cell SSCHA relaxation.
50     """
51     # Read the input structure with CellConstructor.
52     structure = CC.Structure.Structure()
53     structure.read_scf(structure_filename)
54
55     # Convert the CellConstructor structure to an ASE Atoms object so that
56     # ASE optimizers and the QUIP/GAP potential can be used.
57     atoms = Atoms(
58         cell=structure.unit_cell,
59         positions=structure.coords,
60         symbols=structure.atoms,

```

```

61         pbc=True,
62     )
63     atoms.set_calculator(potential)
64
65     # Relax both atomic positions and cell vectors.
66     ecf = ExpCellFilter(atoms)
67     qn = QuasiNewton(ecf)
68     qn.run(fmax=0.0005)
69
70     # Convert the relaxed ASE structure back to CellConstructor format.
71     structure = CC.Structure.Structure()
72     structure.generate_from_ase_atoms(atoms, get_masses=True)
73
74     # Compute an initial dynamical matrix by finite displacements.
75     dyn = CC.Phonons.compute_phonons_finite_displacements(
76         structure,
77         potential,
78         supercell=supercell,
79     )
80     dyn.Symmetrize()
81
82     # Remove imaginary frequencies from the starting guess. This produces a
83     # positive-definite auxiliary dynamical matrix for the SSCHA minimization.
84     dyn.ForcePositiveDefinite()
85
86     # Estimate the static bulk modulus from an equation-of-state fit. This is
87     # needed by the variable-cell SSCHA relaxation below.
88     eos = calculate_eos(atoms)
89     v0, e0, B = eos.fit()
90     bulk = B / kJ * 1.0e24
91
92     return dyn, bulk
93
94
95     # Input files and calculation settings. Adjust these paths for your machine.
96     src = "/scratch/ddangic/sscha_school_2026/Materials/"
97     structure_name = "CsPbI3_cubic_phase.scf"
98     pot = Potential(param_filename=src + "GAP_1.xml", calc_args="local_gap_variance")
99     supercell = [2, 2, 2]
100
101     # Generate the initial auxiliary dynamical matrix and the bulk modulus.
102     dyn, bulk = get_starting_dynamical_matrices(src + structure_name, pot, supercell)
103
104     # SSCHA relaxation settings.
105     temperature = 300.0
106     nconf = 500
107     max_pop = 1000
108
109     # Generate the initial SSCHA ensemble from the starting dynamical matrix.
110     ensemble = sscha.Ensemble.Ensemble(
111         dyn,
112         T0=temperature,
113         supercell=dyn.GetSupercell(),
114     )
115     ensemble.generate(N=nconf)
116
117     # Set up the SSCHA minimizer.
118     minimizer = sscha.SchaMinimizer.SSCHA_Minimizer(ensemble)
119     minimizer.min_step_dyn = 0.001

```

```

120 minimizer.kong_liu_ratio = 0.5
121 minimizer.meaningful_factor = 0.001
122 minimizer.max_ka = 100000
123
124 # Perform the variable-cell SSCHA relaxation.
125 relax = sscha.Relax.SSCHA(
126     minimizer,
127     ase_calculator=pot,
128     N_configs=nconf,
129     max_pop=max_pop,
130     save_ensemble=True,
131 )
132 relax.vc_relax(
133     static_bulk_modulus=bulk,
134     ensemble_loc="directory_of_the_ensemble",
135 )
136
137 # Recompute the SSCHA ensemble with an increasing number of configurations.
138 # This loop is used to test the convergence of the third-order force constants
139 # and the corresponding thermal-conductivity results.
140 for i in range(10):
141     numconf = (i + 1) * 1000
142
143     # Generate a new ensemble using the relaxed SSCHA dynamical matrix.
144     new_ensemble = sscha.Ensemble.Ensemble(
145         relax.minim.dyn,
146         T0=temperature,
147         supercell=relax.minim.dyn.GetSupercell(),
148     )
149     new_ensemble.generate(N=numconf)
150
151     # Compute energies, forces, and stresses for the generated configurations.
152     new_ensemble.compute_ensemble(
153         pot,
154         compute_stress=True,
155         stress_numerical=False,
156         cluster=None,
157         verbose=True,
158     )
159
160     # Minimize the free energy for the new ensemble. We keep the structure fixed
161     # here because the variable-cell relaxation was already performed above.
162     new_minimizer = sscha.SchaMinimizer.SSCHA_Minimizer(new_ensemble)
163     new_minimizer.minim_struct = False
164     new_minimizer.set_minimization_step(0.001)
165     new_minimizer.meaningful_factor = 0.001
166     new_minimizer.max_ka = 10000
167     new_minimizer.init()
168     new_minimizer.run()
169
170     # Save the converged auxiliary dynamical matrix in Quantum ESPRESSO format.
171     new_minimizer.dyn.save_qe("final_dyn_" + str(numconf) + ".")
172
173     # Reweight the ensemble with the updated dynamical matrix.
174     new_ensemble.update_weights(new_minimizer.dyn, temperature)
175
176     # Calculate the free-energy Hessian and the third-order tensor.
177     # Setting return_d3=True returns the third-order force constants needed for
178     # the thermal-conductivity calculation.

```

```

179     dyn_hessian, d3_tensor = new_ensemble.get_free_energy_hessian(
180         include_v4=False,
181         get_full_hessian=True,
182         return_d3=True,
183     )
184
185     # Save the third-order tensor and the Hessian for later use.
186     np.save("d3_" + str(numconf) + ".npz", d3_tensor)
187     dyn_hessian.save_qe("hessian_dyn_" + str(numconf) + ".")

```

In this example, we used a $2 \times 2 \times 2$ supercell to calculate the second- and third-order force constants. The third-order force constants were calculated using an increasing number of configurations in order to test the convergence of the ensemble average.

The second- and third-order force constants must be converged with respect to both the supercell size and the number of configurations. Figure 1 shows the results for harmonic properties. The phonon band structure interpolated from the $2 \times 2 \times 2$ supercell is converged with respect to the number of configurations at around 2000 configurations. However, the results also need to be converged with respect to the supercell size, and they do not appear to be converged even for the $4 \times 4 \times 4$ supercell.

Figure 1. Convergence study of the phonon band structure with respect to supercell size and the number of configurations.

To calculate the thermal conductivity using force constants obtained with different numbers of configurations, we can use the following script:

```

1  from __future__ import division, print_function
2
3  import time
4
5  import numpy as np
6
7  import cellconstructor as CC
8  import cellconstructor.ForceTensor
9  import cellconstructor.Phonons
10 import cellconstructor.ThermalConductivity
11
12 # Number of irreducible q points/files in the saved dynamical matrix.
13 nqirr = 4
14
15 # Useful conversion factors from CellConstructor.
16 SSCHA_TO_MS = cellconstructor.ThermalConductivity.SSCHA_TO_MS
17 RY_TO_THZ = cellconstructor.ThermalConductivity.SSCHA_TO_THZ
18
19 # Use the same grid for phonon properties and scattering processes in this
20 # example. In production calculations, these two grids should be converged
21 # separately.
22 meshnum = 10
23 mesh = [meshnum, meshnum, meshnum]
24
25 # Loop over force constants obtained from different ensemble sizes. This starts
26 # from 2000 configurations because range(1, 10) gives i = 1, ..., 9.
27 for i in range(1, 10):
28     numconf = (i + 1) * 1000
29
30     # Read the auxiliary SSCHA dynamical matrix saved in the previous step.
31     dyn_prefix = "final_dyn_" + str(numconf) + "_"
32     dyn = CC.Phonons.Phonons(dyn_prefix, nqirr)
33     supercell = dyn.GetSupercell()

```

```

34
35 # Create the third-order force-constant object using the same supercell
36 # used to generate the SSCHA dynamical matrix.
37 fc3 = CC.ForceTensor.Tensor3(
38     dyn.structure,
39     dyn.structure.generate_supercell(supercell),
40     supercell,
41 )
42
43 # Load the third-order tensor and use it to initialize fc3.
44 d3_name = "d3_" + str(numconf) + ".numpy"
45 d3 = np.load(d3_name)
46 fc3.SetupFromTensor(d3)
47
48 # Center the third-order force constants. This step is necessary
49 # in order to interpolate 3rd order force constants.
50 fc3 = CC.ThermalConductivity.centering_fc3(fc3)
51
52 # Define the ThermalConductivity object. The kpoint_grid is the grid on which
53 # phonon quantities are calculated on, while scattering_grid controls
54 # the sampling of scattering processes.
55 tc = CC.ThermalConductivity.ThermalConductivity(
56     dyn,
57     fc3,
58     kpoint_grid=mesh,
59     scattering_grid=mesh,
60     smearing_scale=None,
61     smearing_type="adaptive",
62     cp_mode="quantum",
63     off_diag=True,
64 )
65
66 # Calculate the thermal conductivity at 300 K.
67 temperatures = np.linspace(300, 300, 1, dtype=float)
68 start_time = time.time()
69
70 # Compute harmonic quantities such as phonon frequencies and group
71 # velocities, then write them to text files.
72 tc.setup_harmonic_properties()
73 tc.write_harmonic_properties_to_file()
74
75 # Calculate the lattice thermal conductivity in the SRTA approximation.
76 # Lifetimes are written to file because write_lifetimes=True.
77 tc.calculate_kappa(
78     mode="SRTA",
79     temperatures=temperatures,
80     write_lifetimes=True,
81     gauss_smearing=True,
82     offdiag_mode="perturbative",
83     kappa_filename="ConfThermal_conductivity_SRTA_" + str(numconf),
84     lf_method="fortran-LA",
85 )

```

The most important parts of the script are:

- The mesh used to calculate phonon properties is defined to be the same as the mesh used to calculate scattering processes. This is controlled by the variable `mesh`. These grids do not have to be identical. In many cases, `scattering_grid` can be much smaller than `kpoint_grid`. **Both grids must be converged.**
- We use a smearing approach to satisfy energy-conservation laws. There are two methods: constant and

adaptive. For `smearing_type = "constant"`, the smearing value must be provided in **Ry** as an argument to `setup_harmonic_properties`. For adaptive smearing, as used here, the smearing value is different for different phonon modes and is based on the phonon density of states and **q**-point grid density. A global scaling factor can still be provided through `smearing_scale`; `smearing_scale = 1.0` often works well. **The smearing parameters must also be converged.**

- The `off_diag` variable controls whether the calculation includes what is often called *coherent transport*. This contribution can be important for strongly anharmonic materials with significant bunching of phonon modes.
- The function `calculate_kappa` performs most of the transport calculation. Its main options are:
 - `mode` defines the method used to calculate the lattice thermal conductivity. The options are **SRTA**, which corresponds to the Boltzmann transport equation in the single relaxation time approximation, and **GK** (Dangic et al.), which is the Green-Kubo method based on phonon spectral functions rather than phonon lifetimes. These two modes should give similar results in weakly anharmonic materials, but they can differ in strongly anharmonic materials.
 - `gauss_smearing` defines how energy conservation is treated in the self-energy calculation. If **True**, Gaussian functions are used. If **False**, Lorentzian functions are used. With Gaussian smearing, the real part of the self-energy is calculated using the Kramers-Kronig transformation.
 - `offdiag_mode` defines how coherent transport is calculated when `mode = "SRTA"`. The available options are **wigner** (Simoncelli et al.), **gk** (Isaeva et al.), and **perturbative** (Dangic et al.). If `mode = "GK"`, coherent transport is included naturally.
 - `lf_method` defines how phonon lifetimes are calculated when `mode = "SRTA"`. In most cases, keep the `fortran-` prefix and add either **LA** or **P**; these options should give similar results. Another option is **SC**, where phonon lifetimes are solved self-consistently and phonon lineshifts are included.
 - `ne` defines the number of frequency steps used when calculating phonon lineshapes. It is also important for `lf_method = "SC"`, because the self-consistent equation is solved on a frequency grid and the real and imaginary parts of the self-energy are linearly interpolated. Larger values are generally more accurate. **The results must be converged with respect to `ne`.**

In this example, we use the *SRTA* approximation, which is the weak-anharmonicity limit of the Green-Kubo equation. We use SSCHA auxiliary dynamical matrices because they form the basis for the dynamical extension of the SSCHA, derived from time-dependent SSCHA, which gives access to phonon lifetimes and lineshapes.

By setting `off_diag=True`, we include the coherent contribution to the lattice thermal conductivity. This contribution is important in strongly anharmonic materials, alloys, and amorphous solids. The calculation should take a few minutes, and the results are saved using the filename prefix specified by `kappa_filename`.

The thermal conductivity should also be calculated using different **q**-point grids. Figure 2 includes results obtained for a $3 \times 3 \times 3$ grid. These calculations do not appear to be converged at the $3 \times 3 \times 3$ level, so a larger supercell would be required for a production calculation.

However, third-order force constants often converge faster with respect to supercell size than second-order force constants. Therefore, one can sometimes combine dynamical matrices computed on a larger supercell with third-order force constants computed on a smaller supercell. This approach is illustrated in the script below:

```

1 from __future__ import print_function
2 from __future__ import division
3
4 import time
5
6 import numpy as np
7 import cellconstructor as CC
8 import cellconstructor.Phonons
9 import cellconstructor.ForceTensor
10 import cellconstructor.ThermalConductivity
11
12 SSCHA_TO_MS = cellconstructor.ThermalConductivity.SSCHA_TO_MS

```

```

13 RY_TO_THZ = cellconstructor.ThermalConductivity.SSCHA_TO_THZ
14
15 # Define the q-point grid used for both harmonic properties and scattering processes.
16 meshnum = 10
17 mesh = [meshnum, meshnum, meshnum]
18
19 # Define the temperature at which the thermal conductivity will be calculated.
20 temperatures = np.linspace(300, 300, 1, dtype=float)
21
22 # Read the dynamical matrix corresponding to the same supercell used to compute
23 # the third-order force constants.
24 dyn_prefix = "final_dyn_2x2x2_"
25 nqirr = 4
26 dyn_for_fc3 = CC.Phonons.Phonons(dyn_prefix, nqirr)
27
28 # Initialize and load the third-order force-constant tensor.
29 supercell = dyn_for_fc3.GetSupercell()
30 fc3 = CC.ForceTensor.Tensor3(
31     dyn_for_fc3.structure,
32     dyn_for_fc3.structure.generate_supercell(supercell),
33     supercell,
34 )
35
36 d3_name = "d3_2x2x2.npy"
37 d3 = np.load(d3_name)
38 fc3.SetupFromTensor(d3)
39
40 # Center the third-order force constants.
41 fc3 = CC.ThermalConductivity.centering_fc3(fc3)
42
43 # Read a dynamical matrix computed using a larger supercell.
44 # This matrix will provide the harmonic properties used in the transport calculation.
45 dyn_prefix = "./final_dyn_4x4x4_"
46 nqirr = 10
47 dyn = CC.Phonons.Phonons(dyn_prefix, nqirr)
48
49 # Build the ThermalConductivity object using the large-supercell dynamical matrix
50 # and the small-supercell third-order force constants.
51 tc = CC.ThermalConductivity.ThermalConductivity(
52     dyn,
53     fc3,
54     kpoint_grid=mesh,
55     scattering_grid=mesh,
56     smearing_scale=None,
57     smearing_type="adaptive",
58     cp_mode="quantum",
59     off_diag=True,
60 )
61
62 start_time = time.time()
63
64 # Compute harmonic properties and write them to file.
65 tc.setup_harmonic_properties()
66 tc.write_harmonic_properties_to_file()
67
68 # Calculate the thermal conductivity in the SRTA approximation.
69 tc.calculate_kappa(
70     mode="SRTA",
71     temperatures=temperatures,

```

```

72     write_lifetimes=True,
73     gauss_smearing=True,
74     offdiag_mode="perturbative",
75     kappa_filename="Thermal_conductivity_SRTA",
76     lf_method="fortran-LA",
77 )

```

Figure 2. Convergence study of thermal conductivity with respect to q-point sampling, supercell size, and number of configurations.

Finally, we can calculate the lattice thermal conductivity with the Green-Kubo method:

```

1  from __future__ import division, print_function
2
3  import time
4
5  import numpy as np
6
7  import cellconstructor as CC
8  import cellconstructor.ForceTensor
9  import cellconstructor.Phonons
10 import cellconstructor.ThermalConductivity
11
12 # Use the dynamical matrix and third-order force constants obtained with
13 # 10000 configurations.
14 dyn_prefix = "final_dyn_10000_"
15 nqirr = 4
16
17 # Useful conversion factors.
18 SSCHA_TO_MS = cellconstructor.ThermalConductivity.SSCHA_TO_MS
19 RY_TO_THZ = cellconstructor.ThermalConductivity.SSCHA_TO_THZ
20
21 # Read the auxiliary SSCHA dynamical matrix.
22 dyn = CC.Phonons.Phonons(dyn_prefix, nqirr)
23 supercell = dyn.GetSupercell()
24
25 # Build and initialize the third-order force-constant tensor.
26 fc3 = CC.ForceTensor.Tensor3(
27     dyn.structure,
28     dyn.structure.generate_supercell(supercell),
29     supercell,
30 )
31 d3 = np.load("d3_10000.npy")
32 fc3.SetupFromTensor(d3)
33 fc3 = CC.ThermalConductivity.centering_fc3(fc3)
34
35 # Define the q-point grid for the Green-Kubo calculation.
36 meshnum = 10
37 mesh = [meshnum, meshnum, meshnum]
38
39 # Set up the ThermalConductivity object. With mode="GK", the method uses
40 # phonon spectral functions rather than phonon lifetimes.
41 tc = CC.ThermalConductivity.ThermalConductivity(
42     dyn,
43     fc3,
44     kpoint_grid=mesh,
45     scattering_grid=mesh,
46     smearing_scale=1.0,
47     smearing_type="adaptive",

```



```

48     cp_mode="quantum",
49     off_diag=True,
50 )
51
52 # Compute harmonic quantities once. They are reused for all values of ne below.
53 temperatures = np.linspace(300, 300, 1, dtype=float)
54 start_time = time.time()
55 tc.setup_harmonic_properties()
56 tc.write_harmonic_properties_to_file()
57
58 # Converge the Green-Kubo calculation with respect to the number of frequency
59 # points used to integrate overlaps of phonon spectral functions.
60 for i in range(5):
61     ne = i * 2000 + 1000
62
63     tc.calculate_kappa(
64         mode="GK",
65         temperatures=temperatures,
66         write_lifetimes=False,
67         gauss_smearing=False,
68         ne=ne,
69         offdiag_mode="perturbative",
70         kappa_filename="Thermal_conductivity_GK_" + str(ne),
71         lf_method="fortran-LA",
72     )
73
74 # Save the ThermalConductivity object for post-processing.
75 tc.save_pickle()

```

The Green-Kubo calculation introduces an additional convergence parameter: the number of frequency points used to integrate the overlap of phonon spectral functions. The frequency spacing is approximately $2.1\omega_D/n_e$, where ω_D is the highest frequency on the $\mathbf{q}$ -point grid. For an accurate estimate of the integrals, this frequency spacing should be smaller than the smallest phonon linewidth. Therefore, materials with higher thermal conductivity will likely require larger values of n_e .

The SRTA and GK methods usually agree well for weakly anharmonic materials. For strongly anharmonic materials, however, their results can differ significantly.

The final line of the previous script saves the `ThermalConductivity` object, which can later be used to analyze the calculated transport quantities. The snippet below reloads this object and writes the phonon spectral function at Γ :

```

1  from __future__ import division, print_function
2
3  import time
4
5  import matplotlib.gridspec as gridspec
6  import matplotlib.pyplot as plt
7  import numpy as np
8
9  import cellconstructor as CC
10 import cellconstructor.ForceTensor
11 import cellconstructor.Phonons
12 import cellconstructor.ThermalConductivity
13
14 # Reload the ThermalConductivity object saved with tc.save_pickle().
15 tc = CC.ThermalConductivity.load_thermal_conductivity()
16
17 # Temperature-dependent quantities, such as lineshapes, are stored in

```

```

18 # dictionaries. The keys are strings formatted as format(T, ".1f").
19 keys = list(tc.lineshapes.keys())
20
21 # Find the Gamma point in the q-point grid.
22 for iqpt in range(tc.nkpt):
23     if np.linalg.norm(tc.k_points[iqpt]) == 0.0:
24         break
25
26 # Build the frequency grid associated with the saved lineshape data.
27 energies = (
28     np.arange(len(tc.lineshapes[keys[-1]][0, 0]), dtype=float) * tc.delta_omega
29     + tc.delta_omega
30 )
31
32 # Write the phonon spectral function at Gamma to a text file.
33 tc.write_lineshape(
34     "Lineshape_at_Gamma",
35     tc.lineshapes[keys[-1]][iqpt],
36     iqpt,
37     energies,
38     "no",
39 )

```

Temperature-dependent transport properties, such as phonon lifetimes, heat capacities, and phonon lineshapes, are stored as dictionaries with keys corresponding to the temperature T formatted as `format(T, ".1f")`. The example above writes the phonon spectral function at Γ .

Other relevant quantities, such as phonon frequencies and group velocities, can also be accessed from the `ThermalConductivity` object. In addition, the object provides built-in functions for calculating the phonon density of states, phonon spectral conductivity, and phonon Grüneisen parameters in cubic systems

4.2 Exercise

Estimate the lattice thermal conductivity of CsPbI₃ in the cubic $Pm\bar{3}m$ phase.

Before computing the thermal conductivity, first check the dynamical stability of this phase. In particular, determine the temperature range in which the cubic phase is dynamically stable and therefore the thermal-conductivity calculation is physically meaningful.

Then compare the lattice thermal conductivity obtained using:

- the single relaxation time approximation (**SRTA**), and
- the Green-Kubo (**GK**) approach.

Discuss the following points:

1. At what temperature does the cubic $Pm\bar{3}m$ phase become dynamically stable?
2. How does the thermal conductivity depend on temperature?
3. Do the SRTA and GK results agree? If not, which method gives a larger thermal conductivity and why?
4. What does the comparison between SRTA and GK tell you about the degree of anharmonicity in cubic CsPbI₃?

Chapter 5

Hands-on-session 5: EPIq - Anharmonicity in electron-phonon coupling related properties

5.1 Introduction

In this hands-on session we learn how to include anharmonic effects calculated within the SSCHA in the calculation of electron-phonon coupling related properties using [EPIq](#).

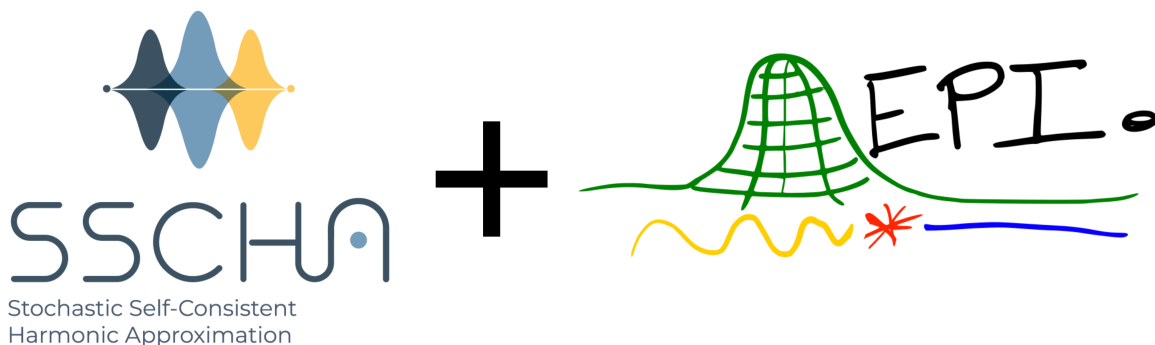


Figure 5.1: logo_epiq

In some systems the first principles calculation of electron-phonon coupling matrix elements can be demanding. EPIq (Electron-Phonon wannier Interpolation over k and q-points) is an open-source software that allows to speed up the calculation of electron-phonon coupling related properties using the Wannier interpolation technique. Details on the interpolation scheme can be found [here](#). Within EPIq, it is possible to include anharmonic corrections to the dynamical matrices as calculated within the SSCHA.

5.2 Requirements

In the interest of time, in this hands-on session the following starting data are at your disposal:

1. Electron-phonon matrix elements $g_{m,n}^\nu(\mathbf{k}, \mathbf{q})$ computed from first principles.
2. Wannier interpolation files which encode the transformation to the optimally smooth subspace, $U_{mn} : |\mathbf{R}n\rangle = \frac{1}{\sqrt{N_k^w}} \sum_{\mathbf{k}=1}^{N_k^w} \sum_{m=1}^{N_w} e^{-i\mathbf{k}\cdot\mathbf{R}} U_{mn}(\mathbf{k}) |\psi_{\mathbf{k}m}\rangle$
3. Anharmonic dynamical matrices $D_{\mu,\nu}^{SCHA}(\mathbf{k}, \mathbf{q})$.
4. Harmonic dynamical matrices (as a reference) $D_{\mu,\nu}^{HARM}(\mathbf{k}, \mathbf{q})$.

Attention

A folder prepared for you with these data for the present tutorial can be downloaded [05_EPIq](#) folder in the shared cloud. Right click on **tutorial_data** on the navigation bar and download the whole folder. It contains:

1. The electron-phonon coupling matrix elements can be found in the **mat_elem** folder. Each file corresponds to a different **q**-point in the first Brillouin zone.
2. The **Wannier** folder contains the files (**.eig**, **.chk**).
3. The dynamical matrices are stored in the **dyn_mat** directory. **dynq*** files are harmonic ($D_{\mu,\nu}^{SCHA}(\mathbf{k}, \mathbf{q})$) while **MoS2.Hessian.dyn*** are anharmonic dynamical matrices computed with the SSCHA code ($D_{\mu,\nu}^{SCHA}(\mathbf{k}, \mathbf{q})$).

Place all the downloaded material where you intend to run the tutorial. A suggested structure is for example:

```

handson_8/ + ---- epiq/ | | | + ---- bin/ | | | + ---- src/ | | + ---- MoS2/ | + ---- MoS2.eig | + ----
MoS2.chk | + ---- mat_elem/ | | | + ---- MoS2_elph.mat.1_q* | + ---- dyn_mat/ | + ---- dynq* | +
---- MoS2.Hessian.dyn*

```

5.3 About EPIq

Epiq site: <https://the-epiq-team.gitlab.io/epiq-site/>

Epiq paper: <https://www.sciencedirect.com/science/article/pii/S0010465523002953>

The Electron-Phonon Interpolation over **q** and **k** points package exploits Wannier interpolation to obtain many proprieties in solids. Bloch theorem allows to describe an infinite system in real space with a continuum of Hamiltonian for different **k**-points in the Brillouin zone.

The properties of a material refers to a certain observable averaged over the sample.

Thanks to the Bloch theorem it is often convenient to perform the average in the reciprocal **k**-space. In other words as a sum over the whole Brillouin zone of the quantities defined at each **k**-point.

$$\langle O \rangle = \frac{1}{N_k} \sum_{\mathbf{k}} F_O(\mathbf{k})$$

The quality of the averaging depends on the finesse of the sampling N_k and of the smoothness of the integrand function $F_O(\mathbf{k})$ (a constant function is totally described with just one **k**-point).

Wannier interpolation is an efficient way to refined the Brillouin zone sampling.

5.3.1 Calculations available in EPIq

1. Adiabatic (static) and non-adiabatic (dynamic) force constant matrices.
2. **Electron-phonon contribution to the phonon linewidth** and related quantities.
3. Isotropic and **anisotropic Eliashberg equations**.
4. Double Resonant Raman scattering.
5. Electron lifetime and relaxation time.

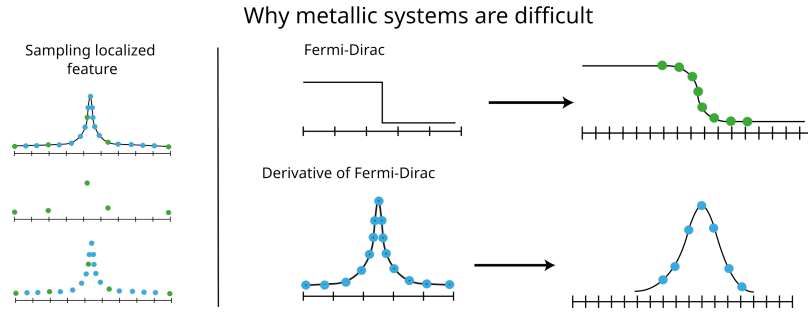


Figure 5.2: sampling

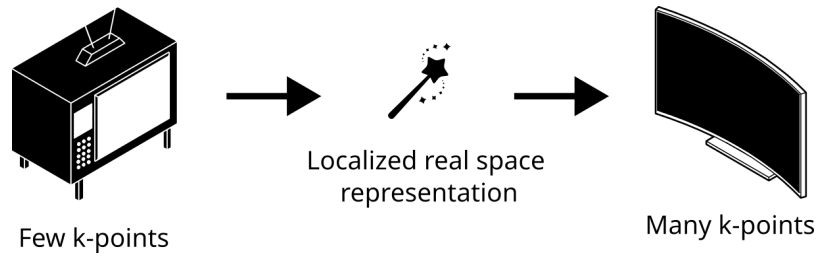


Figure 5.3: wannier_interpolation

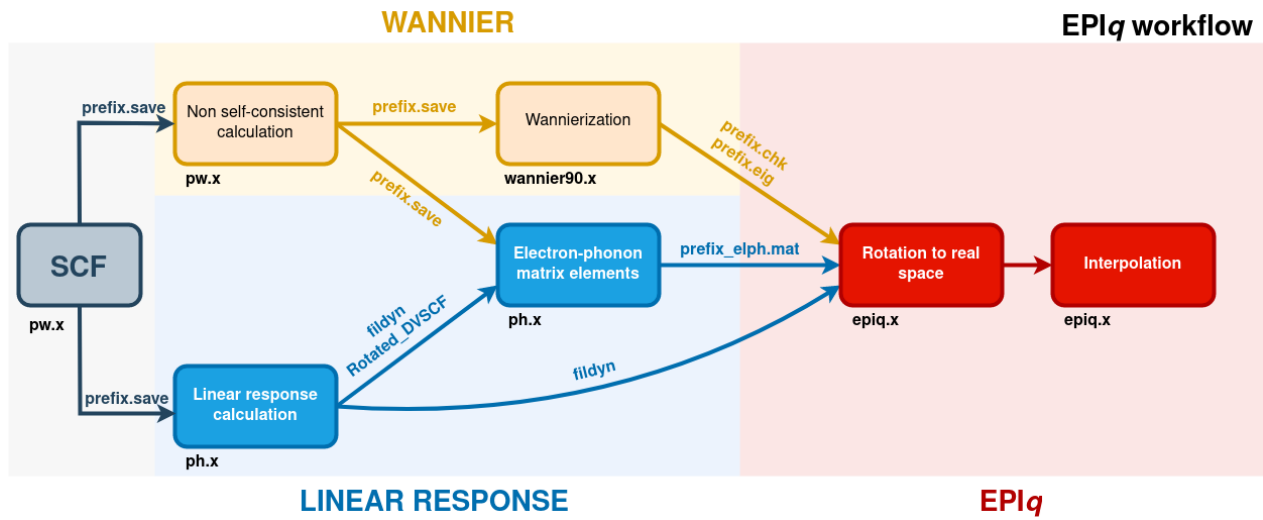


Figure 5.4: wannier

5.3.2 EPIq workflow

The core steps of any calculation employing EPIq are performed using the main executable `epiq.x` and consists in two main stages.

1. A preliminary step where electron-phonon coupling matrix elements and the Hamiltonian are Fourier-transformed to real space and written to file.
2. The electron-phonon coupling matrix elements and the Hamiltonian are transformed back to reciprocal space to compute the property of interest at arbitrary k - and q - values.

5.3.3 EPIq input file

The input file is divided in three namelists:

1. `&control`, specifying what calculation the code will perform
2. `&electrons`, specifying the electronic parameters of the property to be computed
3. `&phonons`, specifying the phonons parameters for the property to be computed

Finally, the last lines of the input indicates the electron momentum (k -) and phonon momentum (q -) meshes on which the matrix elements are interpolated to.

5.4 Setup phase: how to setup a calculation with EPIq

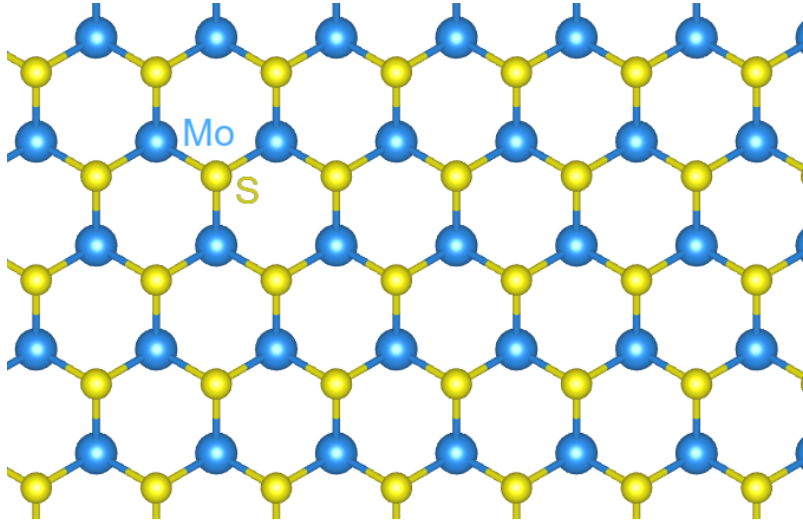


Figure 5.5: wannier

In this tutorial we calculate electron-phonon coupling related properties for doped monolayer MoS_2 , and evaluate the effect of anharmonicity on them.

5.4.1 Wannier interpolation

As explained in the previous section, any calculation within EPIq starts with a preliminary step referred to as `dump`. During this step, the Hamiltonian and the electron-phonon coupling matrix elements in real space are computed and written to file, to be used in any subsequent calculation. This step is performed only once. The input file is the following:

```
1 &control
2   dump_gR=.true.,
3   prefix='MoS2',
4   elphmat_dir="./mat_elem/"
```

```

5 /
6 &electrons
7 /
8 &phonons
9   nq1=8,
10  nq2=8,
11  nq3=1,
12 /

```

Notice, in particular, the following parameters:

- `dump_gR` in the namelist `control` tell the program to save the auxiliary file containing the Hamiltonian and the electron-phonon coupling matrix elements in real space. Since we are not calculating any property of the system the namelist `electron` and `phonons` are essentially empty. Only `nq1,nq2,nq3` have to be supplied in order to specify the q-points mesh where the input electron-phonon coupling matrix elements were computed (in this case, a 8x8x1 q-grid).

Note

In order to keep everything tidy you can use keep the el-ph matrix elements in a separate folder and use the variable:

`&control > elphmat_dir="<path-to-folder>"`

Run this preliminary calculation using:

```

1 mpirun -n <NPROC> {$path_to_epiq}/bin/epiq.x -inp dump.in > dump.out

```

After the dump has ended, some output files have been produced.

- The output file is binary and named `G_and_H.bin` contains the Hamiltonian and the electron-phonon coupling matrix elements in the Wannier representation. If, however, `ascii_G_and_H=.true.` is added in the `&control` namelist then the produced output file is readable and is named `G_and_H.asc`.
- .dat files containing the real space localization of the Hamiltonian and the electron-phonon coupling matrix elements.

5.4.2 Check real space localization

If the Wannier transformation is well converged, the matrix elements are optimally localized in real space. Always check their localization.

Test:

Using the two-columns files:

$$\begin{aligned}
 \text{"MoS2_gw_R_ph.mu*.dat.pe_1"} \quad |R| \quad & \sum_{m,n} |g_{m,n}^\nu(0, \mathbf{R})|^2 \\
 \text{"MoS2_gw_R_el.mu*.dat.pe_1"} \quad |r| \quad & \sum_{m,n} |g_{m,n}^\nu(\mathbf{r}, 0)|^2
 \end{aligned}$$

plot the averaged modulus of the electron-phonon matrix elements as a function of the distance in real space $|R|$. Are they localized?

5.5 Exercise phase: calculation of electron-phonon coupling related properties for doped monolayer MoS₂

Input example for phonon-electron linewidth calculation in monolayer MoS₂

Consider the following input file for the phonon-electron linewidth calculation γ at the M-point of the Brillouin zone. The input parameters are explained in detail in the [EPIq manual](#). Here is the input file:

```

1 &control
2   prefix='MoS2',
3   calculation='ph_linewidth',
4   read_dumped_gr=.true.,
5   dump_gR=.false.,
6   elphmat_dir='./mat_elem/'
7   out2json=.true.
8 /
9
10 &electrons
11   ngauss=0,
12   sigma_min=0.01,
13   sigma_max=0.05
14   nsigma=5,
15 /
16
17 &phonons
18   use_alternative_dyn=.true.
19   prefix_alt_dyn='./dyn_mat/dynq'
20   Fourier_interp_dyn=.true.,
21   nq1=8,nq2=8,nq3=1,
22 /
23
24 k-points
25 automatic
26 4 4 1 0 0 0
27
28 q-points
29 crystal
30 1
31 0.5 0 0 1 ! M

```

The linewidth calculation is then started by:

```

1 mpirun -n <NPROC> {$path_to_epiq}/bin/epiq.x -inp lw.in > lw.out

```

Note that this calculation is done within the harmonic approximation, as we are using the dynamical matrices indicated by the variable `prefix_alt_dyn='./dyn_mat/dynq'`.

Exercise:

Perform a convergence study of the linewidth at **M** as function of the smearing and the k-mesh density. What is the minimum temperature at which the linewidth of the eighth mode is to be considered at convergence with a k-mesh of 8x8x1 ? And of 12x12x1?

Exercise:

Which mode shows larger smearing dependence?

Inclusion of anharmonicity

Now, we want to observe what changes with the inclusion of **anharmonic effects**. To this aim, we need to correctly specify the prefix of the Free-energy Hessian matrices calculated within the SSCHA using `prefix_alt_dyn='./dyn_mat/MoS2.Hessian.dyn'`.

Exercise:

Compute the linewidth at **q = M** using anharmonic dynamical matrix computed using SSCHA. Why does it seem like the first and the second mode are exchanged with respect to the harmonic dynamical matrices? What phonon mode(s) shows largest anharmonic correction?

Exercise:

Try to produce two plots of the whole phonon spectrum, comparing the harmonic and the anharmonic result. Do you observe any differences?

Tip: a plottable file can be obtained using the `linewidth_path.x` post-processing tool. Here is an example input file:

```
1 &input_lambda
2   prefix='MoS2'
3   lkp_sequential=.true.
4   sigma_min=0.01,
5   sigma_max=0.02,
6   nsigma=2,
7   chosen_sigma=0.01
8 &end
9   3.159998    0.000000    0.000000
10  -1.579999    2.736639    0.000000
11   0.000000    0.000000    19.141895
12 crystal
13 11
14 0.001 0 0 1
15 0.05 0 0 1
16 0.10 0 0 1
17 ...
18 ...
19 0.50 0 0 1
```

Notice the parameter `chosen_sigma`, which specifies what smearing will be used to produce the plottable file, and the lattice parameters at the end of the namelist. The post-processing is executed as follows:

```
1 {$path_to_epiql}/bin/linewidth_path.x < path.in > path.out
```

You can then use the following gnuplot script plots the q-resolved linewidth for the acoustic modes:

```
1 set ylabel '{/Symbol w}(meV)'
2 set xlabel 'Gamma - M'
3 set style fill transparent solid 0.25
4 pl for [i=0:9] 'MoS2_lw_path.d' every 9::i u 1:2 w l lt rgb 'black' title ''
5 repl for [i=0:9] 'MoS2_lw_path.d' every 9::i u ($1):($2-$3/2):($2+$3/2)\
6 w filledc lt rgb 'red' title ''
```

5.6 Solution: Phonon linewidth calculation

Let's start again from the example file we considered for the calculation for phonon of momentum $\mathbf{q} = \mathbf{M}$ for two values of the electronic smearing.

```
1 &control
2   prefix='MoS2',
3   calculation='ph_linewidth',
4   read_dumped_gr=.true.,
5   dump_gR=.false.,
6   elphmat_dir='./mat_elem/'
7   out2json=.true.
8 /
9
10 &electrons
11   ngauss=0,
```

```

12     sigma_min=0.01,
13     sigma_max=0.05
14     nsigma=5,
15 /
16
17 &phonons
18     use_alternative_dyn=.true.
19     prefix_alt_dyn='./dyn_mat/dynq'
20     Fourier_interp_dyn=.true.,
21     nq1=8,nq2=8,nq3=1,
22 /
23
24 k-points
25 automatic
26 4 4 1 0 0 0
27
28 q-points
29 crystal
30 1
31 0.5 0 0 1 ! M

```

The phonon linewidth $\gamma_{\mathbf{q},\nu}$ in the double delta (Allen) form is defined by the following equation:

$$\gamma_{\mathbf{q},\nu} = \frac{4\pi\omega_{\mathbf{q},\nu}}{N_k} \sum_{m,n} \sum_{\mathbf{k}} |g_{m,n}^{\nu}(\mathbf{k}, \mathbf{q})|^2 \delta(\epsilon_{\mathbf{k}+\mathbf{q},m} - \epsilon_F) \delta(\epsilon_{\mathbf{k},n} - \epsilon_F)$$

where the electron-phonon coupling is defined from the deformation potential as:

$$g_{m,n}^{\nu}(\mathbf{k}, \mathbf{q}) = \sum_s \mathbf{e}_{\mathbf{q},\nu}^s \cdot \mathbf{d}_{m,n}^s(\mathbf{k}, \mathbf{q}) / \sqrt{2M_s\omega_{\mathbf{q},\nu}}$$

Parameters

- The linewidth calculation is selected by setting the calculation parameter equal to 'ph_linewidth' in the &control namelist.
- In the namelist control, read_dumped_gr, which is set to .true., indicating that electron-phonon coupling matrix elements can be read from G_and_H.bin)
- In the namelist electrons, sigma_min, sigma_max, ngauss and nsigma specify maximum, minimum, type and number of electronic smearing values to use. efermi and ef_from_input specify the initial guess for the Fermi level (the Fermi level calculated by Quantum ESPRESSO is usually a good guess) and whether the Fermi level should be re-calculated by epiq (ef_from_input equal to .false.) or set from input (ef_from_input equal to .true.). The variable thr_compute_k is used to restrict the calculation only to k-points possessing at least one eigenvalue in the specified energy region near the Fermi level.
- In the namelist phonons, Fourier_interp_dyn equal to .true. asks to interpolate the dynamical matrices for the q-points that do not belong to the Wannier grid. Alternatively, *EPIq* gives the opportunity to read eigenvalues and eigenvectors produced by matdyn.x of the Quantum ESPRESSO package (matdyn.eig file), putting Fourier_interp_dyn=.false. and read_modes=.true. in input, or even to directly read dynamical matrices from a matdyn.dyn file, putting Fourier_interp_dyn=.false. and read_modes=.false. .

Output files

If the out2json flat is set to .true., the file MoS2_lambda.json will be produced. It can be automatically parsed using python as in the following lines where the variable q_pts is a "dict" whose entries are the results of the calculation for each q-point.

```

1 import json
2 ff = './MoS2_lambda.json'
3 with open(ff,'r') as f:
4     q_pts = json.load(f)
5
6 first_q = q_pts["1"]
7
8 print("Fraction coordinate of the q point:", first_q["xq_frac"])
9 print("Electronic temperatures used:", first_q["T"])
10 print("Results for the first mode:", first_q["1"])
11 print("Frequency of the second mode in meV:", first_q["2"]["freq"])

```

Here, a simple python script to plot the linewidth esteemed with the Allen formula:

```

1 import matplotlib.pyplot as plt
2 import numpy as np
3 for q in list(q_pts.values())[:]:
4     for mod in range(1,10):
5         plt.plot( q["T"],q[f'{mod}']["gamma_allen"],label=r"$\omega_{"+f'{mod}'+f"={q[f'{mod}']["freq"]:.0f} (meV)",linestyle='--' )
6     plt.xlabel('T (eV)')
7     plt.ylabel(r'$\gamma(T)$ (meV)')
8     plt.legend(title=f"q={ np.round(np.array(q['xq_frac']),2) }")
9     plt.show()

```

The other output file, MoS2_lambda.d, contains all the properties calculated by EPIq. The way this file is formatted is specified in output file, lw.out. Notice in particular that the first column contains the electronic smearing, while the second column contains the Allen linewidth.

Dispersion along a line

Now we want to perform the calculation of phonon linewidth along a certain crystalline direction and produce a plot like this one:

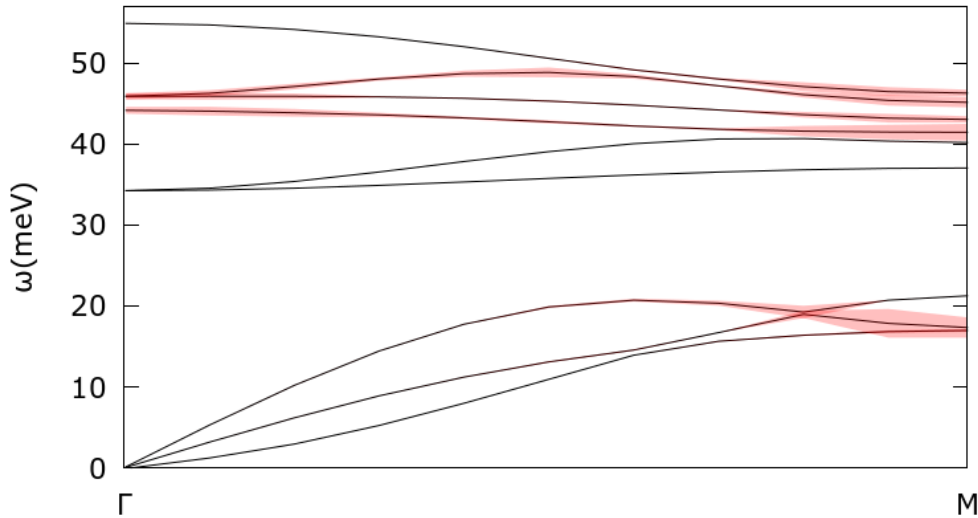


Figure 5.6: wannier

where the thickness of the lines is proportional to the calculated phonon linewidth. In order to to this, we repeat the phonon linewidth calculation, this time considering the whole Γ -M path:

```

1  &control
2      dump_gr=.false.,
3      read_dumped_gr=.true.,
4      prefix='MoS2',
5      calculation='ph_linewidth',
6      elphmat_dir="./mat_elem/"
7  &end
8  &electrons
9      ngauss=0,
10     sigma_min=0.01,
11     sigma_max=0.02
12     nsigma=2,
13 &end
14 &phonons
15     use_alternative_dyn=.true.
16     prefix_alt_dyn='./dyn_mat/MoS2.Hessian.dyn'
17     Fourier_interp_dyn=.true.,
18     nq1=8,nq2=8,nq3=1,
19 &end
20 k-points
21 automatic
22 8 8 1 0 0 0
23 q-points
24 crystal
25 11
26 0.001 0 0 1
27 0.05 0 0 1
28 0.10 0 0 1
29 ...
30 ...
31 0.50 0 0 1

```

Once the linewidth calculation has finished, we can obtain a plottable file using the `linewidth_path.x` post-processing tool. Here is an example of input file:

```

1  &input_lambda
2      prefix='MoS2'
3      lkp_sequential=.true.
4      sigma_min=0.01,
5      sigma_max=0.02,
6      nsigma=2,
7      chosen_sigma=0.01
8  &end
9      3.159998    0.000000    0.000000
10     -1.579999    2.736639    0.000000
11     0.000000    0.000000    19.141895
12 crystal
13 11
14 0.001 0 0 1
15 0.05 0 0 1
16 0.10 0 0 1
17 ...
18 ...
19 0.50 0 0 1

```

Notice the parameter `chosen_sigma`, which specifies what smearing will be used to produce the plottable file, and the lattice parameters at the end of the namelist. The post-processing is executed as follows:

```
1 { $path_to_epiq }/bin/linewidth_path.x < path.in > path.out
```

5.7 Advanced tutorial: Migdal-Eliashberg calculation using SS-CHA Hessian matrices

EPIq also allows to solve the anisotropic Eliashberg equations on the imaginary axis in order to calculate the $\mathbf{k}$ -resolved superconducting gap. We give an example for the superconducting gap of doped monolayer MoS₂ at T= 1 K.

```
1 &control
2   dump_gR=.false.,
3   read_dumped_gr=.true.,
4   prefix='MoS2',
5   calculation='migdal_eliashberg',
6   elphmat_dir="./mat_elem/"
7 &end
8 &electrons
9   theta=1,
10  efermi=-2.0,
11  ef_from_input=.false.,
12 &end
13 &phonons
14   use_alternative_dyn=.false.
15   prefix_alt_dyn='./dyn_mat/MoS2.Hessian.dyn'
16   read_modes=.false.,
17   nq1=8,
18   nq2=8,
19   nq3=1,
20 &end
21 &input_migdal
22   initialize='step',
23   gap_threshold=0.025,
24   sigma_me=0.1,
25   ME_Fermi_thickness=0.4,
26   mustar=0.1,
27   nmatsu=64,
28   nitermax=100,
29   alpha_mix_me=0.5,
30   gap_init=4.0,
31 &end
32 nkfs
33 64 64 1
34 40
```

Run the *EPIq* calculation as follows:

```
1 mpirun -n 4 { $path_to_epiq }/bin/epiq.x <input_ME > out_ME
```

Note that for the `migdal_eliashberg` calculation, the number of $\mathbf{k}$ -points (40 in this case) must be a multiple of the mpi processes (4 in this case).

Notice the following parameters in the input file:

- In the `&electrons` namelist, `theta=1.0` specifies the temperature of the calculation in Kelvin.
- In the `&input_migdal` namelist, `sigma_me` specifies the broadening (in eV) to be used in the calculation and `ME_Fermi_thickness` the energy range around the Fermi level where electron eigenvalues are considered

in the calculation. `nmatsu` indicates the number of Matsubara frequencies to be employed in the sum (see [here](#) for further details:)

- The code solves the equation generating a certain number of random $\mathbf{k}$ -points, defined by the `64 64 1` grid in input and having an eigenvalue on the Fermi surface.

EPIq also calculates the Fermi surface using the grid specified after `nkfs` (`64 64 1` here). After EPIq is done, two output files are produced:

- `MgB2.bxsf` contains the Fermi surface in the XCrysDen `.bxsf` format.
- `MgB2_ME.d` contains the Fermi-surface resolved Eliashberg gap.

You can produce a plot of the Fermi-surface resolved superconducting gap using the `plot_ME_fs.x` post processing. Execute it as:

```
1 { $path_to_epiq }/bin/plot_ME_fs.x
```

This is, for example, what you get for MgB_2 , with its famous double gap:

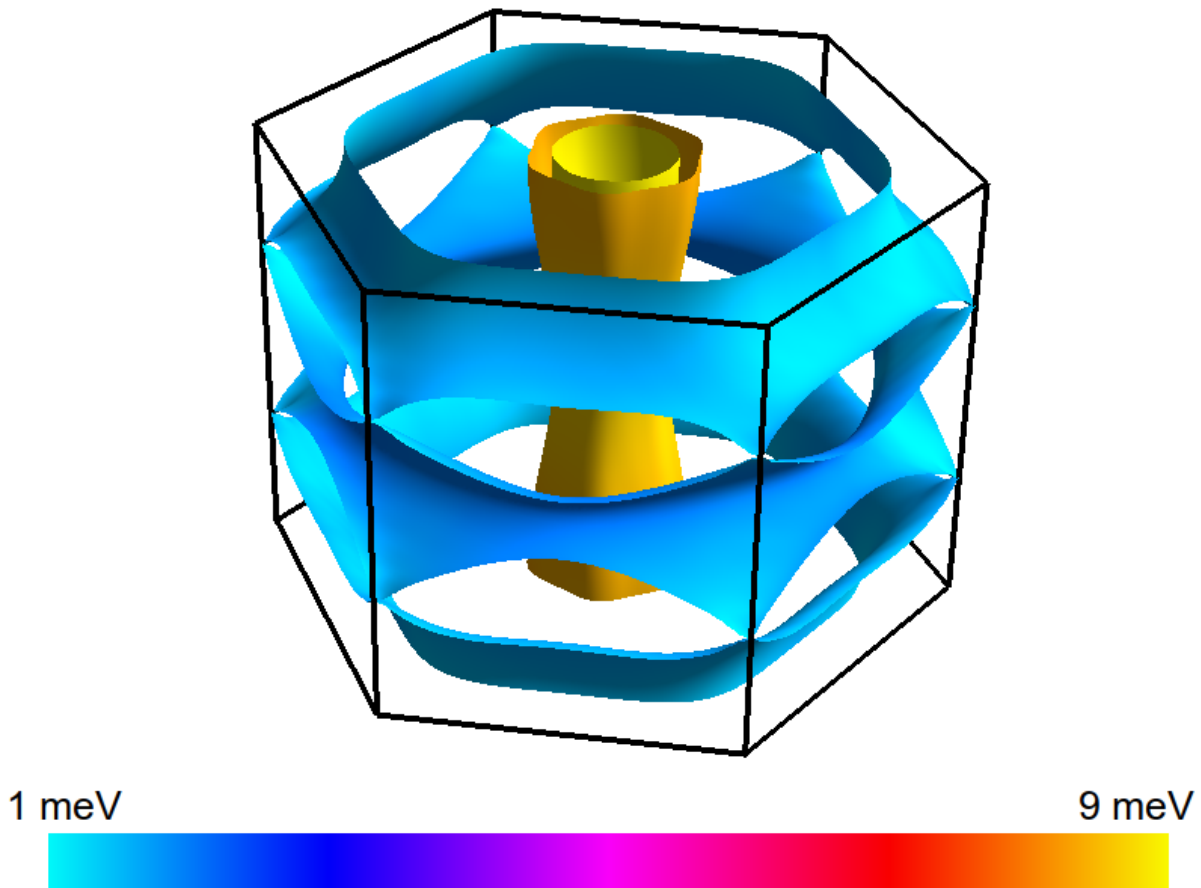


Figure 5.7: `logo_epiq`

Try to obtain the same kind of plot for MoS_2 using `fermisurfer` ([github repository](#)) typing:

```
1 fermisurfer MoS2.frmsf
```

Exercise:

Perform a convergence study of the average superconducting gap as function of the number of Matsubara frequencies and k-points. What is the required value of `nmatsu` and k-points to have a precision better than 0.1 meV?

Note

A complete calculation of anharmonic electron phonon coupling related properties within SSCHA+EPIq requires the following steps:

1. [The SSCHA code](#) → First, compute the free energy Hessian within the stochastic self-consistent harmonic approximation (SSCHA).
2. [Quantum ESPRESSO code](#) → Then, compute electron-phonon coupling matrix elements following the instructions reported in the EPIq site <https://the-epiq-team.gitlab.io/epiq-site/docs/tutorials/step1/>.
3. [Wannier90 code](#) → Identify the unitary transformation connecting the Bloch and the maximally-smooth gauge (required to interpolate the electron-phonon coupling matrix elements in the Wannier basis).
4. [EPIq code](#) → Perform the electron-phonon coupling interpolation in the Wannier basis and calculate physical properties including the anharmonic correction from SSCHA.

All these open-source software can be downloaded following the instruction in each website.

If you want to know more about the full procedure, please take a look at the tutorials on the [EPIq site](#).